

MPlookup: A Quasilinear Multi-Party Lookup Argument in Collaborative zkSNARKs via Four Sorts and a Multi-Point Evaluation

Huayi Qi
qi@huayi.email
Institute for Network Sciences and
Cyberspace, Tsinghua University
Beijing, China

Minghui Xu
mhxu@sdu.edu.cn
School of Computer Science and
Technology, Shandong University
Qingdao, Shandong, China

Tingchuang Zhang
tczhang@mail.sdu.edu.cn
School of Computer Science and
Technology, Shandong University
Qingdao, Shandong, China

Xiuzhen Cheng
xzcheng@sdu.edu.cn
School of Computer Science and
Technology, Shandong University
Qingdao, Shandong, China

Zhijun Li
richikun2014@gmail.com
School of Computer Science and
Technology, Shandong University
Qingdao, Shandong, China

Chao Zhang*
chaoz@tsinghua.edu.cn
Institute for Network Sciences and
Cyberspace, Tsinghua University
Beijing, China

Abstract

A lookup argument is a cryptographic primitive that allows a prover to convince verifiers that every element of a private query vector belongs to a public table vector without disclosing the underlying data. It can enforce correct instruction execution in zero-knowledge virtual machines and serve as an important supplement to zero-knowledge succinct non-interactive arguments of knowledge (zkSNARKs). However, existing lookup argument protocols operate exclusively in the single-prover setting and do not address the requirements of collaborative zkSNARKs, in which multiple parties jointly generate proofs over additively secret-shared data while preserving privacy from both other provers and verifiers.

This work presents MPlookup, the first multi-party lookup argument protocol for collaborative zkSNARKs. MPlookup achieves quasilinear $O(N \log^2 N)$ complexity through four oblivious sorting operations together with a multi-point polynomial evaluation performed entirely over secret shares. We introduce a multi-point evaluation protocol in the distributed oblivious polynomial evaluation setting, constructed via oblivious subproduct tree construction and oblivious polynomial division with private divisors. We prove that the protocol satisfies obliviousness, completeness, soundness, and zero-knowledge. We implement MPlookup as an open-source Rust library, built upon the collaborative zkSNARKs framework and the CompatCircuit arithmetic black box. Our evaluation confirms a performance improvement over an $O(N^2)$ baseline while remaining competitive given the obliviousness requirement.

CCS Concepts

• **Security and privacy** → **Privacy-preserving protocols**; *Security protocols*.

Keywords

lookup argument, collaborative zkSNARKs, oblivious multi-point evaluation, secure multi-party computation, non-interactive zero-knowledge proof

1 Introduction

A lookup argument [8, 19, 26, 36, 39, 41, 46–48, 50], also known as lookup table argument, is a non-interactive zero-knowledge (NIZK) protocol that allows a prover to convince verifiers that every element of a private query vector is contained within a public table vector, without revealing any information beyond the validity of the statement. Lookup arguments are essential for enforcing the correct execution of instructions and are widely used in zero-knowledge virtual machines (zkVMs) [2, 29, 45, 49], serving as a critical primitive alongside zkSNARKs [9, 20, 35].

Recently, Ozdemir et al. [33] introduced collaborative zkSNARKs [1, 16, 22, 31, 33, 37], a significant new research direction. In this setting, instead of a single prover, multiple provers holding partial private inputs collaboratively convince verifiers of a statement’s validity. This is achieved using additive-share-based multi-party computation (MPC) to ensure that no information is leaked to either the *provers* or the verifiers. Collaborative zkSNARKs enable applications where no single party should have access to all sensitive data, such as collective healthcare statistics [15], credit scoring [12],

Table 1: Comparison of lookup argument schemes.

Scheme	Multi-Party?	$\mathcal{T}_{\text{precomp}}$	$\mathcal{T}_{\text{prove}}$	$\mathcal{T}_{\text{verify}}$
Plookup [19]	No	-	$O((N + M) \log(N + M))$	$O(1)$
Halo2 [48]	No	-	$O(N \log N)$	$O(1)$
logUp [34]	No	-	$O(N \log N)$	$O(1)$
Caulk [46]	No	$O(N \log N)$	$O(M^2 + M \log N)$	$O(1)$
Baloo [47]	No	$O(N \log N)$	$O(M \log^2 M)$	$O(1)$
Lasso [39]	No	-	$O(N + M)$	$O(\log^2 M)$
Flookup [18]	No	$O(N \log N)$	$O(N \log^2 N)$	$O(1)$
Our strawman	Yes, n parties	-	$O(N^2)$	$O(1)$
Our MPlookup	Yes, n parties	-	$O(N \log^2 N)$	$O(1)$

Denote N as the length of table vector, M as the length of query vector.

*Corresponding author.

and private multi-party audits [38]. Due to the complexities of combining non-linear MPC protocols with SNARKs, collaborative zkSNARKs still lack essential supporting tools. Qi et al. [37] built a collaborative zkVM, which suffered from performance degradation due to the absence of a multi-party lookup argument protocol.

Table 1 demonstrates the comparison of lookup argument schemes. All existing lookup argument schemes [19, 39, 46–48] focus on a plaintext prover setting, unsuitable for the collaborative zkSNARK setting. This motivates our core research question:

Is it feasible to construct an oblivious multi-party lookup argument protocol, and at what time complexity?

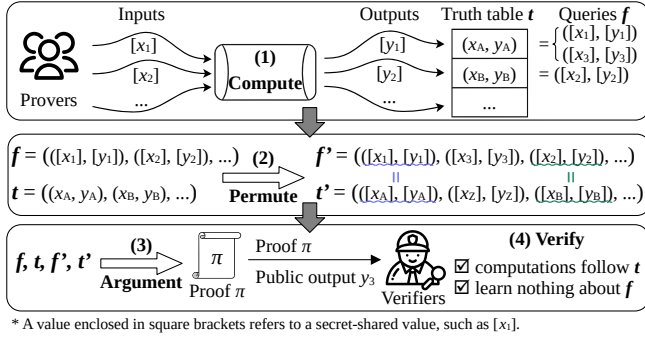


Figure 1: Overview of a multi-party lookup argument.

Challenge I. Making lookup argument preprocessing efficient under secret sharing. Figure 1 illustrates the lookup argument scenario in four steps. **(1) Compute.** Provers jointly evaluate a function on private inputs: for each secret input $[x_i]$, they run a compute protocol and obtain a secret-shared output $[y_i]$. A pre-existing public truth table t enumerates every valid input-output pair, so correctness reduces to a membership claim—every queried pair $([x_i], [y_i])$ must appear *somewhere* in t . **(2) Permute.** All queried pairs are assembled into a secret query vector f . The provers permute f and t into aligned vectors f' and t' , placing each queried pair at the same index as its matching table entry. Repeated queries, i.e., queries that duplicate a value already seen, are paired with unused table entries as placeholders. This *preprocessing* step is the core of the challenge: f is secret-shared, so no prover may observe its actual contents during the permutation. **(3) Argument.** Using the four aligned vectors, the provers run a collaborative NIZK protocol that produces a compact proof π together with any intended public outputs. **(4) Verify.** Any verifier checks π and becomes convinced that every computation was consistent with t , without learning anything about the private inputs beyond the disclosed outputs. In the plaintext single-prover setting, preprocessing takes $O(N \log N)$ via sorting and looping with a hash table. In the MPC setting, f is secret-shared. To ensure obliviousness that no prover can ever see the actual query values, we first propose a naive strawman approach, shown in Figure 2. This strawman algorithm tests all element pairs through oblivious equality checks via inner loops, incurring $O(N^2)$ MPC operations, which is prohibitively expensive.

Challenge II. Efficient multi-point evaluation in distributed oblivious polynomial evaluation. The efficiency of our MPlookup protocol depends on a secure multi-point evaluation protocol with

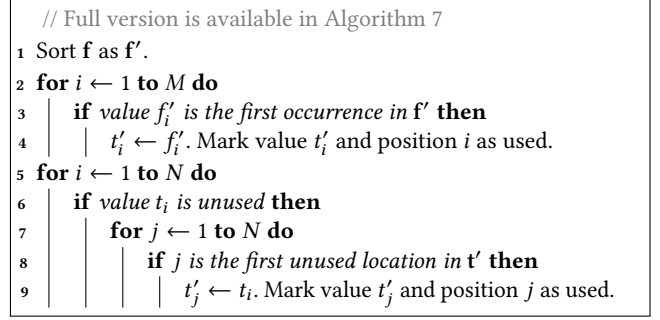


Figure 2: Strawman $O(N^2)$ lookup argument preprocessing, omitting MPC implementation details.

$O(N \log^2 N)$ complexity. However, to the best of our knowledge, no such protocol currently exists for the distributed oblivious polynomial evaluation (OPE) setting. In standard OPE, a sender holds a d -degree polynomial $Q(\cdot)$ while a receiver holds an element x and receives the result $Q(x)$. Distributed OPE extends this to cases where the input is secret-shared among n parties. Frikkien [17] utilizes only single-point evaluation, resulting in a quadratic number of operations. Similarly, while Dong et al. [13] acknowledge the need for multi-point evaluations on encrypted polynomials, they do not provide a concrete construction. Bhadauria et al. [6] suggest potential applications for multi-point evaluation, but as it was not the primary focus of their work, they do not offer a detailed protocol. Finally, while collaborative zkSNARKs [33] support multi-point evaluation when inputs are roots of unity, MPlookup requires evaluation on arbitrary inputs.

Our contributions are listed as follows.

- We are the first to define and study the lookup argument problem in the multi-party setting, and to propose concrete protocols for it. As an initial baseline, we introduce a strawman $O(N^2)$ protocol that directly realizes the required preprocessing constraints via nested oblivious equality checks, establishing the first concrete multi-party lookup argument. Then, we propose MPlookup, the first efficient n -party lookup argument protocol for collaborative zkSNARKs, achieving $O(N \log^2 N)$ complexity. MPlookup allows a group of provers to generate an NIZK proof without leaking information to either the provers or the verifiers. We utilize four oblivious sorts and a multi-point evaluation for an N -degree polynomial over a finite field to compute f' and t' obliviously. Our performance and security analyses confirm that MPlookup achieves $O(N \log^2 N)$ complexity while satisfying obliviousness, completeness, soundness, and zero-knowledge.

- We propose an n -party multi-point evaluation protocol for the distributed OPE setting, supporting evaluation on arbitrary inputs, instead of roots of unity only. To achieve this, we construct n -party versions of subproduct tree generation from secret points, polynomial division with a secret divisor, and division-based multi-point evaluation, all while maintaining full obliviousness. The protocol achieves $O(N \log^2 N)$ complexity and serves as a vital component of our preprocessing algorithm.

- We provide a complete, open-source implementation of the MPlookup protocol, including the multi-point evaluation component. The system is implemented in Rust, building upon the collaborative zkSNARKs framework [33] with the ABB re-implemented from CompatCircuit [37]. Evaluation results show that our protocol offers a moderate performance improvement over the quadratic strawman. While it naturally incurs more overhead than a single-prover plaintext protocol, the computation and communication costs are reasonable given the security guarantees of the oblivious setting. Our code is available at <https://github.com/MPlookup/mplookup-artifacts>.

Organization. Section 2 reviews related work. Section 3 introduces the system model, threat model, and preliminaries. Section 4 presents MPlookup. Section 5 provides formal performance and security analyses. Section 6 describes the implementation and experimental evaluation. Section 7 discusses further optimizations and extensions. Section 8 concludes our work.

2 Related Work

Lookup argument schemes. All existing lookup argument schemes focus on a plaintext prover setting. To the best of our knowledge, our MPlookup is the first lookup argument focusing on a multi-party oblivious setting. Gabizon et al. introduced Plookup [19], a simplified polynomial protocol for lookup tables that proves subset membership by reducing it to a multiset equality check over sorted sequences. The Halo2 proof system [48] by Zcash constructs a lookup PIOP inspired by offline memory checking protocols, reducing the global subset check into local constraints. Building on this direction, Papini et al. proposed logUp [34], which replaces the grand-product multiset check with logarithmic-derivative identities computed using GKR, reducing costly group-element commitments.

A second line of work reduces the prover’s dependence on the full table size N through table-specific preprocessing. Zapico et al. introduced Caulk [46], the first sublinear lookup argument in N , leveraging KZG-based subvector aggregation at the cost of $O(N \log N)$ preprocessing. Posen et al. proposed Caulk+ [36], which simplifies Caulk’s construction and reduces the prover cost to $O(M^2)$ while retaining sublinearity in the table size. Zapico et al. then proposed Baloo [47], reducing the prover time to $O(M \log^2 M)$ via a matrix-vector extraction technique. Gabizon et al. proposed flookup [18], achieving quasi-linear prover time through a pairing-based accumulation approach with fractional decomposition.

More recent work has introduced new paradigms and broader extensions. Setty et al. proposed Lasso [39], a lookup argument over structured and decomposable tables achieving $O(N + M)$ prover time with no table-specific preprocessing. Zhang et al. [50] extended the cq line of work with an efficient KZG-based univariate sum-check, achieving $O(M \log M)$ prover time independent of table size. Campanelli et al. [8] further extended the lookup argument framework with improvements and applications to zero-knowledge decision trees. Tran et al. proposed LogaLookup [41], a multivariate lookup argument for accelerated proof generation. Hafezi et al. [26] provide a systematic survey of lookup table arguments, unifying and comparing the main construction paradigms.

Polynomial multi-point evaluation. MPlookup requires an oblivious multi-point evaluation protocol. In the plaintext setting,

the classical subproduct-tree algorithm achieves $O(N \log^2 N)$ operations for a univariate polynomial over N arbitrary points [28]. Recent work has pushed these bounds further. Bhargava et al. [7] give a deterministic $(d^m + N)^{1+o(1)}$ algorithm for *multivariate* multipoint evaluation over all finite fields $\mathbb{F}_q$. Ghosh et al. [23] extend near-linear multivariate evaluation to infinite fields via finite-field lifting and fast Chinese remaindering [7]. Van der Hoeven and Lecerf [42] tighten deterministic complexity bounds for univariate and low-variate evaluation over any field by analyzing the pre-computation cost of amortized evaluation algorithms. All of these algorithms operate in the plaintext setting and therefore cannot be applied directly in our multi-party setting.

In the MPC setting, prior work on oblivious polynomial evaluation does not provide an $O(N \log^2 N)$ multi-point protocol. Frikken [17] relies exclusively on single-point evaluation, resulting in $O(N^2)$ total operations when applied N times. Dong et al. [13] acknowledge the need for multi-point evaluations on encrypted polynomials but do not provide a concrete construction. Bhaduria et al. [6] suggest potential applications for multi-point evaluation but do not offer a detailed protocol. Collaborative zkSNARKs [33] support multi-point evaluation, but only when the evaluation points are roots of unity, whereas MPlookup requires evaluation at arbitrary inputs. The closest work in the MPC setting is Giorgi et al. [24], who improve the quadratic Mohassel–Franklin protocols for secure polynomial operations over a finite field $\mathbb{F}_q$, achieving $O(\tau n^{1+1/\tau})$ secure multiplications and $O(\tau)$ rounds for multi-point evaluation, where τ is a tunable constant. For any fixed constant τ , $n^{1+1/\tau}$ grows faster than $N \log^2 N$. Moreover, this work does not have an implementation.

3 Model and Preliminaries

This section introduces the system model, threat model, and the cryptographic primitives on which MPlookup is built.

3.1 System Model and Threat Model

We consider a system with three role types for MPlookup: provers $P_1, P_2, \dots, P_n$, non-designated verifiers, and a trusted initializer. The n provers jointly hold a private query vector $\mathbf{f}$, distributed among them as secret shares¹ over a prime field $\mathbb{F}_p$. The table vector $\mathbf{t}$ is public and known to all parties.

The goal of the provers is to collaboratively generate a non-interactive zero-knowledge proof attesting that every element of $\mathbf{f}$ belongs to $\mathbf{t}$, without leaking any information about $\mathbf{f}$ to other provers or to the verifiers. Verifiers check the validity of the proof and accept or reject it, while learning nothing about $\mathbf{f}$ beyond the truth of the membership statement. The trusted initializer performs a trusted setup to produce the public parameters for the underlying collaborative zkSNARK as well as preshared triples for MPC.

We primarily focus on the semi-honest security model, and will provide a malicious security model discussion in Section 7.0.2. A static passive adversary may corrupt up to t of the n provers. A corrupted prover follows the prescribed protocol faithfully but attempts to infer information about the honest provers’ private

¹ Given that the lookup argument serves to validate the correctness of a computation, the elements of input $\mathbf{f}$ are secret shares derived from prior computations, rather than as plaintext inputs attributed to a specific party.

Functionality $\mathcal{F}_{\text{ABB}}$

$\mathcal{F}_{\text{Input}}$: On input (Input, P_i , id, x) from P_i and (Input, P_i , id) from all other parties, with id a fresh identifier and $x \in \mathbb{F}_p$, store (id, x).
 $\mathcal{F}_{\text{Add}}$: On input (Add, id, id_1 , id_2) from all parties, retrieve (id₁, x), (id₂, y), compute $z = x + y$ modulo p , and store (id, z).
 $\mathcal{F}_{\text{Mult}}$: On input (Mult, id, id_1 , id_2) from all parties, retrieve (id₁, x), (id₂, y), compute $z = x \cdot y$ modulo p , and store (id, z).
 $\mathcal{F}_{\text{Eq}}$: On input (Eq, id, id_1 , id_2) from all parties, retrieve (id₁, x), (id₂, y), compute $z = 1$ if $x = y$, 0 otherwise, and store (id, z).
 $\mathcal{F}_{\text{LT}}$: On input (LT, id, id_1 , id_2) from all parties, retrieve (id₁, x), (id₂, y), compute $z = 1$ if $x < y$, 0 otherwise, and store (id, z).
 $\mathcal{F}_{\text{IsZero}}$: On input (IsZero, id, id_1) from all parties, retrieve (id₁, x), compute $y = 1$ if $x = 0$, 0 otherwise, and store (id, y).
 $\mathcal{F}_{\text{Inverse}}$: On input (Inverse, id, id_1) from all parties, retrieve (id₁, x), compute $y = x^{-1}$ if $x \neq 0$, 0 otherwise, and store (id, y).
 $\mathcal{F}_{\text{Open}}$: On input (Output, id) from all honest parties, retrieve (id, y) and output it to the adversary. Wait for an input from the adversary; if this is Deliver then output y to all parties, otherwise output Abort.

Figure 3: Ideal functionality for the MPC arithmetic black-box model in $\mathbb{F}_p$.

data from its own local view of the computation. The protocol is required to reveal no information about the private inputs of honest provers to any collusion of at most t corrupted parties. In addition, if provers are able to produce a false proof, they no longer have the privacy consideration and all of them can collude and stop following the protocol to produce a false NIZK proof.

MPlookup is designed to achieve the following four security properties. A formal definition will be presented in Section 5.2.

(1) *Completeness*. A multi-party lookup argument protocol is complete if, whenever $\mathbf{f} \subseteq \mathbf{t}$ and all provers are honest, the collaboratively generated proof π is accepted by the verifier with probability 1.

(2) *Soundness*. A multi-party lookup argument protocol is sound if, whenever $\mathbf{f} \not\subseteq \mathbf{t}$, provers can produce a proof π accepted by the verifier with a negligible probability.

(3) *Obliviousness*. A multi-party lookup argument protocol is oblivious if corrupted provers learn nothing about $\mathbf{f}$ beyond the truth of the membership statement, assuming there are at most t corrupted provers.

(4) *Zero-knowledge*. A multi-party lookup argument protocol is zero-knowledge if a verifier learns nothing about $\mathbf{f}$ beyond the truth of the membership statement.

3.2 Preliminaries

We introduce the key cryptographic building blocks used throughout the paper: collaborative zkSNARKs, the arithmetic black-box abstraction, and the Halo2 lookup argument.

3.2.1 Collaborative zkSNARKs. A zkSNARK is a cryptographic protocol in which a prover convinces a verifier that a pair $(x, w) \in R$ holds for an NP relation R , without disclosing the witness w . Here, x denotes the public instance and w denotes the private witness. Classical zkSNARK constructions [10, 20, 25, 32] support only a single prover. Ozdemir et al. [33] extended zkSNARKs to the multi-prover setting, introducing collaborative zkSNARKs. Subsequent work [1, 16, 22, 31] has further improved their efficiency and security guarantees. In a collaborative zkSNARK, n provers $P_1, P_2, \dots, P_n$ each hold a witness share w_i , and they collectively convince a verifier $\mathcal{V}$ that $(x, w_1, \dots, w_n) \in R$ through multi-party proof generation. The scheme consists of three algorithms:

- **Setup**($1^\lambda, R$) $\rightarrow$ pp: Run by the trusted initializer; outputs public parameters pp.

- **Prove**(pp, $x, w_1, \dots, w_n$) $\rightarrow \pi$: Run jointly by the n provers; produces a proof π when $(x, w_1, \dots, w_n) \in R$, and aborts otherwise.
- **Verify**(pp, x, π) $\rightarrow \{0, 1\}$: Run by the verifier; returns 1 if and only if π is a valid proof for x .

An (n, t) collaborative zkSNARK satisfies properties below:

- **Completeness**: Honest provers holding valid witness shares satisfying $(x, w_1, \dots, w_n) \in R$ always produce a valid proof.
- **Knowledge soundness**: Any computationally bounded provers that convince the verifier must collectively encode a valid witness, except with negligible probability in the security parameter λ .
- **t -zero-knowledge**: No collusion of at most t corrupted provers or verifiers learns any information about the witnesses $w_1, w_2, \dots, w_n$ beyond the fact that the statement is true.
- **Succinctness**: Both the proof size and the verification time are $o(|R|)$, where $|R|$ denotes the size of relation R .

3.2.2 Arithmetic Black-Box. Secure multi-party computation [5, 11, 27] allows a group of parties to evaluate a function over their private inputs while revealing nothing beyond the designated output. MPlookup builds on additive secret sharing over $\mathbb{F}_p$: each secret value $x \in \mathbb{F}_p$ is split into n shares $x_1, \dots, x_n$ such that $x_1 + \dots + x_n \equiv x \pmod{p}$, with prover P_i holding share x_i .

We abstract the multi-party computation layer as an arithmetic black box, whose ideal functionality $\mathcal{F}_{\text{ABB}}$ is defined in Figure 3.

Figure 3 defines the ideal functionality $\mathcal{F}_{\text{ABB}}$ for an arithmetic black box in the MPC setting. The ABB provides a set of operations, including input sharing, addition, multiplication, equality testing, comparison, zero testing, inversion, and output opening. A prover utilizes these operations to perform computations on secret-shared data without revealing any information to other provers or verifiers.

3.2.3 Halo2 lookup argument. The Halo2 lookup argument allows a prover, given a private query vector $\mathbf{f} = (f_1, f_2, \dots, f_M)$ and a public table vector $\mathbf{t} = (t_1, t_2, \dots, t_N)$ with $M \leq N$, to convince verifiers that every element of $\mathbf{f}$ belongs to $\mathbf{t}$. The protocol proceeds in three phases: setup, proof generation, and verification.

Setup. A trusted initializer runs a universal setup to generate public parameters for a KZG polynomial commitment scheme over a multiplicative subgroup H of order L , where L is the smallest power of two satisfying $L \geq N + \kappa + 1$, where κ is the number of blinding rows. A committer key ck and a verifier key vk are

derived from the public parameters and distributed to the prover and verifiers.

Proof generation. The prover runs as follows:

(1) *Preprocessing* $\mathcal{F}_{\text{Preprocess}}$. The prover first constructs permuted vectors $\mathbf{f}'$ and $\mathbf{t}'$ satisfying the following 3 constraints.

CONSTRAINT 3.1. $\mathbf{f}'$ is a permutation of $\mathbf{f}$.

CONSTRAINT 3.2. $\mathbf{t}'$ is a permutation of $\mathbf{t}$.

CONSTRAINT 3.3. For each index $i \in [1, N]$, if f'_i is the first occurrence of its value in $\mathbf{f}'$, then $f'_i = t'_i$.

(2) *Permutation and vanishing arguments* $\mathcal{F}_{\text{PermVanish}}$. The prover encodes $\mathbf{f}, \mathbf{t}, \mathbf{f}', \mathbf{t}'$ as univariate polynomials over H and computes a grand-product polynomial $Z(X)$ using random challenges β, γ that certifies the multiset relationship $\mathbf{f} \cup \mathbf{t} \approx \mathbf{f}' \cup \mathbf{t}'$. A combined constraint polynomial $C(X)$ encodes the first-occurrence alignment and the grand-product condition; the prover then produces a KZG proof that $C(X)$ vanishes on H .

Verify. The verifier replays the Fiat-Shamir transcript to recover the challenges, verifies the KZG opening proofs, and checks that the evaluated constraint polynomial $C(\zeta)$ equals the claimed quotient $\hat{Q}$ times the vanishing polynomial $Z_H(\zeta) = \zeta^L - 1$.

4 MPlookup Design

This section presents the design of MPlookup, covering an overview of the protocol, the core preprocessing algorithm, the oblivious multi-point evaluation protocol, and the full proof generation and verification procedure.

4.1 Overview

MPlookup builds upon the single-prover Halo2 lookup argument [48] and lifts it to the collaborative zkSNARK setting. Among these procedures, the *preprocessing* step in proof generation is the most challenging part.

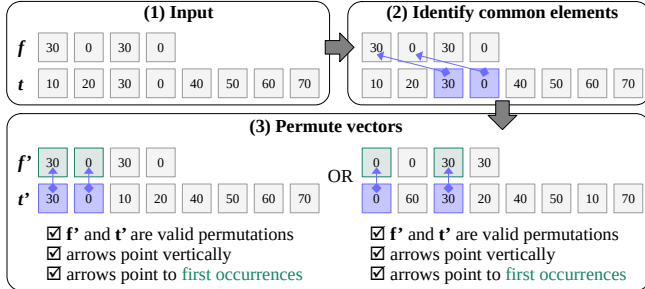


Figure 4: Illustration of ideal preprocessing steps

Figure 4 illustrates the preprocessing goal with a small example, where each element encodes a pair (x_i, y_i) as a single field value $\alpha \cdot x_i + y_i$. (1) **Input.** Query vector $\mathbf{f} = (30, 0, 30, 0)$ and table $\mathbf{t} = (10, 20, 30, 0, 40, 50, 60, 70)$. (2) **Identify common elements.** The values 30 and 0 appear in both vectors. Arrows connect each such element in $\mathbf{t}$ to its counterpart in $\mathbf{f}$, but the arrows are not vertical, which means the matching elements sit at *different* indices in the two vectors. (3) **Permute.** The vectors are permuted into

$\mathbf{f}'$ and $\mathbf{t}'$ satisfying the three constraints annotated in the figure, making all arrows vertical and pointing to first occurrences in $\mathbf{f}'$. Two valid outcomes are shown. The challenge ahead is to carry out this permutation *obliviously*—without any prover learning the actual contents of $\mathbf{f}$.

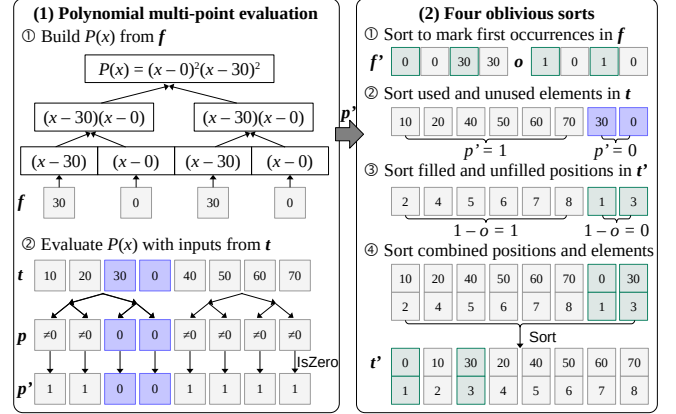


Figure 5: Illustration of MPlookup preprocessing

Figure 5 gives an overview of our core ideas that enable the efficient $O(N \log^2 N)$ oblivious preprocessing. The left half shows how the query set is encoded as a membership polynomial $P(x)$ whose roots are the queried values; evaluating P at every table entry in parallel identifies, obviously, which table elements are queried and which are unused. The right half shows how four oblivious sort operations on carefully constructed composite keys then merge these membership bits with position indices to produce the aligned vectors $\mathbf{f}'$ and $\mathbf{t}'$ required by the lookup argument.

(1) **Polynomial multi-point evaluation.** This step performs membership test. Rather than testing every query–table pair individually, we encode the entire query set as a single polynomial $P(x) = \prod_{i=1}^M (x - f'_i)$, whose roots are exactly the distinct values in the sorted query vector $\mathbf{f}'$. Evaluating P at all N table entries simultaneously via an oblivious multi-point evaluation yields a boolean vector $\mathbf{p}'$ in the end, where $p'_j = 0$ means t_j is queried and $p'_j = 1$ means t_j is unused. Both building P and evaluating it cost $O(N \log^2 N)$, with all data remaining secret-shared throughout.

(2) **Four oblivious sorts.** With $\mathbf{p}'$ in hand, four oblivious sorts complete the alignment. The first sort produces the sorted query vector $\mathbf{f}'$ and marks first-occurrence positions as indicator $\mathbf{o}$. The second sort ranks table entries by $\mathbf{p}'$, bringing queried entries to the front. The third sort ranks $\mathbf{t}'$ positions by the unfilled-position indicator $1 - \mathbf{o}$, bringing empty slots to the front. The fourth sort merges both ranked lists by position index, placing each unused table element into an empty slot and each first-occurrence query value into its designated slot, producing $\mathbf{t}'$. Each sort costs $O(N \log^2 N)$, giving the total $O(N \log^2 N)$ complexity for MPlookup preprocessing.

Migrating to an efficient multi-party proof generation. In the collaborative zkSNARK setting, provers hold $\mathbf{f}$ in secret shares while $\mathbf{t}$ is public.

(1) *Preprocessing.* The ideal functionality $\mathcal{F}_{\text{Preprocess}}$ for this step is defined in Figure 6. It accepts $(\mathbf{f}, \mathbf{t})$ from all parties and returns

Functionality $\mathcal{F}_{\text{Preprocess}}$

Permute input vectors: On input $(\mathbf{f}, \mathbf{t})$ from all parties, prepare output $(\mathbf{f}', \mathbf{t}')$ as instructed in Figure 2.

Functionality $\mathcal{F}_{\text{PermVanish}}$

Permutation and vanishing arguments: On input secret-shared vectors $(\mathbf{f}', \mathbf{t}', \mathbf{f})$ from all parties together with public table $\mathbf{t}$ and public parameters $(L, H, \kappa, \text{ck})$:

- Encode $\mathbf{f}, \mathbf{t}, \mathbf{f}', \mathbf{t}'$ as polynomials over H with κ blinding rows; compute and publish KZG commitments; derive Fiat-Shamir challenges β, γ .
- Compute the grand-product polynomial $Z(X)$; publish Z_{Com} ; derive challenge ζ .
- Compute the quotient polynomial $Q(X) = C(X)/Z_H(X)$; publish Q_{Com} .
- Open all polynomials at ζ , together with Z at $\omega\zeta$ and $\mathbf{f}'$ at $\omega^{-1}\zeta$; publish evaluations and KZG opening proofs.

Output the public proof component $\pi_{\text{perm}} = (f_{\text{Com}}, t_{\text{Com}}, f'_{\text{Com}}, t'_{\text{Com}}, Z_{\text{Com}}, Q_{\text{Com}}, \hat{f}, \hat{t}, \hat{f}', \hat{t}', \hat{Z}, \hat{Z}_{\omega}, \hat{f}_{\omega^{-1}}, \hat{Q}, \pi_{\zeta}, \pi_{\omega\zeta}, \pi_{\omega^{-1}\zeta})$.

Figure 6: Ideal functionality for MPlookup proof generation. The provers first permute the input vectors, then generate the NIZK proof via permutation and vanishing arguments.

$(\mathbf{f}', \mathbf{t}')$ satisfying the three alignment properties stated above. Although in the single-prover setting this requires only a sort and a linear hash-set scan, the multi-party setting demands fully oblivious computation: the output $(\mathbf{f}', \mathbf{t}')$ must be constructed from secret-shared $\mathbf{f}$ and public $\mathbf{t}$ without revealing any intermediate value. Our realization of this functionality achieves $O(N \log^2 N)$ complexity via four oblivious sort operations and an oblivious multi-point polynomial evaluation on arbitrary inputs, replacing the $O(N^2)$ nested-loop approach of the strawman.

(2) *Permutation and vanishing arguments.* The ideal functionality $\mathcal{F}_{\text{PermVanish}}$ for this step is defined in Figure 6. The required techniques, including arithmetic operations, polynomial operations with public divisors, and evaluation on roots of unity, are covered by the ABB and existing collaborative zkSNARK techniques [33], so this step migrates directly to the oblivious setting.

4.2 Multi-Point Evaluation based Lookup Argument Preprocessing Algorithm

We present the core MPlookup preprocessing algorithm, which obviously permutes the query and table vectors in $O(N \log^2 N)$ time using multi-point evaluation and four oblivious sorts.

We show our MPlookup preprocessing protocol in Algorithm 1. We now illustrate the execution of Algorithm 1 with example vectors

$$\mathbf{f} = (30, 0, 30, 0) \quad \mathbf{t} = (10, 20, 30, 0, 40, 50, 60, 70)$$

where $N = 8$ and $M = 4$. Numbers in blue highlight the first-occurrence elements for illustrative purposes only. All vectors are shown in plaintext for illustrative purposes; in practice, every operation is performed on secret-shared data.

Algorithm 1: MPlookup Preprocess $\Pi_{\text{Preprocess}}$

Input: Secret-shared query vector $\mathbf{f}$ and public table vector $\mathbf{t}$, where $M = |\mathbf{f}| \leq N = |\mathbf{t}|$. $\mathbf{t}$ contains unique elements, whereas $\mathbf{f}$ may contain duplicates.

Output: Secret-shared permuted vectors $\mathbf{f}'$ and $\mathbf{t}'$ satisfying Constraint 3.1, Constraint 3.2, and Constraint 3.3

```

1  $\mathbf{f}' \leftarrow \text{Sort}(\mathbf{f})$ 
2  $\mathbf{f}'' \leftarrow \mathbf{f}' \parallel \mathbf{0}^{N-M}$ 
   // 1. Oblivious sort on  $\mathbf{f}$  and pad to size  $N$  ( $O(N \log^2 N)$ )
3  $\mathbf{o} \leftarrow (1) \parallel \{1 - \text{Eq}(f'_i, f'_{i-1})\}_{i=2}^M \parallel \mathbf{0}^{N-M}$ 
   // 2. Identify first occurrence elements in  $\mathbf{f}'$  via adjacent comparisons and pad to size  $N$  ( $O(N)$ )
4 Compute coefficients of  $P(x) \leftarrow \prod_{i=1}^M (x - f'_i)$  by building a subproduct tree from  $\mathbf{f}$ 
   // 3. Construct polynomial  $P(x)$  that returns 0 if and only if  $x \in \mathbf{f}'$  ( $O(M \log M)$ )
5 Build a subproduct tree from  $\mathbf{t}$  and compute  $\mathbf{p} \leftarrow (P(t_1), P(t_2), \dots, P(t_N))$  via oblivious multi-point evaluation
   // 4. Evaluate  $P(x)$  at each elements in  $\mathbf{t}$  ( $O(N \log^2 N)$ )
6  $\mathbf{p}' \leftarrow \{1 - \text{IsZero}(p_j)\}_{j=1}^N$ 
   // 5. Normalize vector  $\mathbf{p}$  to a boolean vector ( $O(N)$ )
7  $\mathbf{r} \leftarrow \{(j, t_j, p'_j)\}_{j=1}^N$ 
8  $\mathbf{r}' \leftarrow \text{Sort}(\mathbf{r})$  ordered by  $r_j[3] = p'_j$  descending, then by  $r_j[1] = j$  ascending
   // 6. Construct and sort a vector of unused elements, their positions, and valid bits in  $\mathbf{t}$  ( $O(N \log^2 N)$ )
9  $\mathbf{s} \leftarrow \{(i, 1 - o_i)\}_{i=1}^N$ 
10  $\mathbf{s}' \leftarrow \text{Sort}(\mathbf{s})$  ordered by  $s_i[2] = 1 - o_i$  descending, then by  $s_i[1] = i$  ascending
   // 7. Construct and sort a vector of unfilled positions and valid bits in  $\mathbf{t}'$  ( $O(N \log^2 N)$ )
11  $\mathbf{l}^{(1)} \leftarrow \{(s'_i[1], r'_i[2], s'_i[2])\}_{i=1}^N$ 
12  $\mathbf{l}^{(2)} \leftarrow \{(i, f''_i, o_i)\}_{i=1}^N$ 
13  $\mathbf{l} \leftarrow \mathbf{l}^{(1)} \parallel \mathbf{l}^{(2)}$ 
14  $\mathbf{l}' \leftarrow \text{Sort}(\mathbf{l})$  ordered by  $l_i[3]$  descending, then by  $l_i[1]$  ascending
   // 8. Create and sort a vector combining all the positions, the values, and valid bits ( $O(N \log^2 N)$ )
15  $\mathbf{t}' \leftarrow \{l'_i[2]\}_{i=1}^N$ 
16 return  $\mathbf{f}', \mathbf{t}'$ 
   // 9. The first  $N$  elements in  $\mathbf{l}'$  are the outputs. Discarding the rest  $N$  elements in  $\mathbf{l}'$  ( $O(N)$ )

```

Step 1. Oblivious sort on $\mathbf{f}$ and padding to size N yields

$$\mathbf{f} = (0, 0, 30, 30), \quad \mathbf{f}'' = (0, 0, 30, 30, 0, 0, 0, 0).$$

This step groups identical query elements adjacently and extends the vector to size N . We use 0 as the padding value here but any values can be used.

Step 2. Identification of first-occurrence positions via adjacent equality comparisons produces the indicator vector

$$\mathbf{o} = (1, 0, 1, 0, 0, 0, 0, 0).$$

Here, $o_1 = 1$ by definition, and $o_i = 1 - \text{Eq}(f'_i, f'_{i-1})$ for $i = 2, 3, 4$. Positions $i = 1$ with value 0 and $i = 3$ with value 30 are the first occurrences of each distinct element in the padded query vector.

Step 3. The polynomial $P(x) = \prod_{i=1}^M (x - f'_i) = (x - 0)^2 (x - 30)^2$ is constructed via an oblivious subproduct tree over the elements of $\mathbf{f}'$. This polynomial evaluates to zero only at the queried values, enabling membership testing against the table.

Step 4. Multi-point evaluation of $P(x)$ at every element of $\mathbf{t}$.

$$\mathbf{p} = (P(10), P(20), P(30), P(0), P(40), P(50), P(60), P(70)),$$

where $P(10) \neq 0$, $P(20) \neq 0$, $P(30) = 0$, $P(0) = 0$, and $P(t_j) \neq 0$ for $j = 5, \dots, 8$.

Step 5. Normalization to a boolean vector via zero-testing yields

$$\mathbf{p}' = (1, 1, 0, 0, 1, 1, 1, 1).$$

Thus, $p'_j = 1$ if and only if t_j does not appear in $\mathbf{f}$, i.e., t_j is unused.

Step 6. The vector $\mathbf{r} = \{(j, t_j, p'_j)\}_{j=1}^N$ is formed and sorted by the valid bit p'_j descending, then by index j ascending, producing

$$\mathbf{r}' = ((1, 10, 1), (2, 20, 1), (5, 40, 1), (6, 50, 1), (7, 60, 1), (8, 70, 1), (3, 30, 0), (4, 0, 0)).$$

The first six entries correspond to unused table elements; the last two correspond to the queried values present in $\mathbf{t}$.

Step 7. The vector of unfilled positions $\mathbf{s} = \{(i, 1 - o_i)\}_{i=1}^N$ is formed and sorted by the unfilled bit $1 - o_i$ descending, then by index i ascending, producing

$$\mathbf{s}' = ((2, 1), (4, 1), (5, 1), (6, 1), (7, 1), (8, 1), (1, 0), (3, 0)).$$

The first six entries identify the unused positions in $\mathbf{t}'$.

Step 8. The combined vector is constructed as

$$\mathbf{I}^{(1)} = \{(s'_i[1], r'_i[2], s'_i[2])\}_{i=1}^N, \quad \mathbf{I}^{(2)} = \{(i, f'_i, o_i)\}_{i=1}^N,$$

followed by concatenation $\mathbf{I} = \mathbf{I}^{(1)} \parallel \mathbf{I}^{(2)}$ and sorting by the third component descending, then by the first component ascending. The resulting $\mathbf{I}'$ begins with the eight triples whose third component equals 1, and there is exactly one per position in $[1, N]$ ordered by position index:

$$\mathbf{I}' = ((1, 0, 1)^{(2)}, (2, 10, 1)^{(1)}, (3, 30, 1)^{(2)}, (4, 20, 1)^{(1)}, (5, 40, 1)^{(1)}, (6, 50, 1)^{(1)}, (7, 60, 1)^{(1)}, (8, 70, 1)^{(1)}, (1, 0, 0)^{(1)}, (3, 30, 0)^{(1)}, (2, 0, 0)^{(2)}, (4, 30, 0)^{(2)}, (5, 0, 0)^{(2)}, (6, 0, 0)^{(2)}, (7, 0, 0)^{(2)}, (8, 0, 0)^{(2)}).$$

We use superscripts (1) and (2) to illustrate whether an element in $\mathbf{I}'$ is from $\mathbf{I}^{(1)}$ or $\mathbf{I}^{(2)}$. This sorting merges the original first-occurrence query values (from $\mathbf{I}^{(2)}$) at their respective positions with the unused table values (from $\mathbf{I}^{(1)}$) at the unfilled positions.

Step 9. The second entries, i.e., the value components, of the first N elements of $\mathbf{I}'$ are extracted. The construction yields vectors

$$\mathbf{t}' = (0, 10, 30, 20, 40, 50, 60, 70),$$

while $\mathbf{f}'$ retains its value from Step 1.

The final output vectors are therefore

$$\mathbf{f}' = (0, 0, 30, 30), \quad \mathbf{t}' = (0, 10, 30, 20, 40, 50, 60, 70).$$

satisfying Constraint 3.1, Constraint 3.2, and Constraint 3.3.

Having demonstrated Algorithm 1, we now claim its correctness. The proof of Theorem 4.1 will be provided in Appendix D.

THEOREM 4.1. *Assuming every element of $\mathbf{f}$ belongs to $\mathbf{t}$, and $\mathbf{t}$ contains unique elements, Algorithm 1 correctly produces output $(\mathbf{f}', \mathbf{t}')$ satisfying Constraint 3.1, Constraint 3.2, and Constraint 3.3.*

4.3 Multi-Point Evaluation in MPC

The efficiency of MPlookup preprocessing, as described in Algorithm 1, relies on an efficient multi-point evaluation protocol to evaluate the membership polynomial $P(x)$ at all table elements.

We now propose an n -party multi-point evaluation protocol in the distributed oblivious setting. This protocol is built from oblivious subproduct trees, oblivious polynomial multiplication based on number theoretic transform (NTT), and an oblivious polynomial division routine, achieving $O(N \log^2 N)$ complexity while relying exclusively on the arithmetic black-box operations without any data-dependent branching.

Oblivious polynomial multiplication. We adopt an existing MPC implementation for polynomial multiplication [33], constructed using NTT over a radix-2 evaluation domain whose size is a power of two. Given two secret-shared coefficient vectors, NTT is applied to each, followed by pointwise secure multiplications and the inverse NTT (INTT).

Algorithm 2: Oblivious Polynomial Inversion

Input: Secret-shared coefficients $f = (f_0, \dots, f_{d-1})$ of a degree- d polynomial with $f_0 \neq 0$

Output: Secret-shared coefficients of the inverse $g(x)$ such that $g(x) \cdot f(x) \equiv 1 \pmod{x^d}$

```

1  $g \leftarrow [f_0^{-1}]$ 
2 for  $\ell = 1$  to  $\lceil \log_2 d \rceil$  do
3    $k \leftarrow 2^\ell$ 
4    $\tilde{f} \leftarrow f[0..k-1]$  padded with zeros to length  $k$ 
5    $h \leftarrow \text{PolyMul}(g, \tilde{f})$ , blindly truncated to length  $k$ 
6    $h \leftarrow \text{PolyMul}(g, h)$ , blindly truncated to length  $k$ 
7    $g' \leftarrow [2g_0, \dots, 2g_{\frac{k}{2}-1}, 0, \dots, 0]$ , of length  $k$ 
8    $g \leftarrow g' - h$ 
9 return  $g[0..d-1]$ 
```

Algorithm 3: Oblivious Polynomial Division

Input: Secret-shared numerator coefficients num and denominator coefficients den (degree $d_{\text{den}} \leq d_{\text{num}}$)

Output: Secret-shared quotient q and remainder r

```

1  $\text{rev\_num} \leftarrow \text{num}$  array reversed
2  $\text{rev\_den} \leftarrow \text{den}$  array reversed
3  $\text{deg}_q \leftarrow d_{\text{num}} - d_{\text{den}} + 1$ 
4  $\text{inv\_rev\_den} \leftarrow \text{PolyInv}(\text{rev\_den}, \text{deg}_q)$ 
5  $\text{q\_rev} \leftarrow \text{PolyMul}(\text{rev\_num}[0..\text{deg}_q-1], \text{inv\_rev\_den})$ ,
   blindly truncated to length  $\text{deg}_q$ 
6  $q \leftarrow \text{q\_rev}$  array reversed
7  $\text{q\_den} \leftarrow \text{PolyMul}(q, \text{den})$  padded to length  $M+1$ 
8  $r \leftarrow \text{num} - \text{q\_den}$ 
9 return  $(q, r)$ 
```

Oblivious polynomial inversion and division. To enable division within the oblivious setting, we first construct Algorithm 2 for polynomial inversion via Newton iteration, which iteratively doubles the approximation degree using only field additions and multiplications. Given the inversion routine, polynomial division with remainder as shown in Algorithm 3 is performed algebraically via the reversal technique, again using only oblivious multiplications, additions, and the polynomial inverse. For a numerator of degree d_{num} and a denominator of degree $d_{\text{den}} \leq d_{\text{num}}$, the quotient and remainder are obtained without any secret-dependent control flow.

Algorithm 4: Oblivious Subproduct Tree Construction

Input: Secret-shared points $p_0, \dots, p_{k-1}$
Output: Subproduct tree $\mathcal{T}$ with ℓ levels

```

1 leaf  $\leftarrow [(-p_i, 1) \text{ for each } i]$ 
2 Compute  $k'$  to the next power of two,  $k' \geq k$ 
3 Pad leaf to length  $k'$  with polynomial 1
4  $\mathcal{T}[0] \leftarrow \text{leaf}$ 
5 cur  $\leftarrow \text{leaf}$ 
6 for  $\ell = 1$  to  $\log_2 k'$  do
7   next  $\leftarrow []$ 
8   for  $i = 0$  to  $\lceil \frac{|cur|}{2} \rceil - 1$  do
9      $p_1 \leftarrow \text{cur}[2i]$ 
10     $p_2 \leftarrow \text{cur}[2i + 1]$  if  $2i + 1 < |cur|$ , 1 otherwise
11    next  $\leftarrow \text{next} \cup [\text{PolyMul}(p_1, p_2)]$ 
12    $\mathcal{T}[\ell] \leftarrow \text{next}$ 
13   cur  $\leftarrow \text{next}$ 
14 return  $\mathcal{T}$ 
```

Algorithm 5: Oblivious Multi-Point Evaluation

Input: Secret-shared polynomial coefficients p (degree $< 2^\ell$), subproduct tree $\mathcal{T}$ with ℓ levels
Output: Secret-shared evaluations $p(p_0), \dots, p(p_{2^\ell-1})$

```

1 reduced[lvl][0]  $\leftarrow p$ 
2 for lvl =  $\ell - 1$  downto 0 do
3   foreach parent node  $j$  at level lvl + 1 do
4     cur  $\leftarrow \text{reduced}[lvl + 1][j]$ 
5     if left child exists then
6       child  $\leftarrow \mathcal{T}[lvl][2j]$ 
7       ( $\_$ , rem)  $\leftarrow \text{PolyDivMod}(\text{cur}, \text{child})$ 
8       Blindly truncate rem to length  $2^{\text{lvl}}$ 
9       reduced[lvl][2j]  $\leftarrow \text{rem}$ 
10    if right child exists then
11      child  $\leftarrow \mathcal{T}[lvl][2j + 1]$ 
12      ( $\_$ , rem)  $\leftarrow \text{PolyDivMod}(\text{cur}, \text{child})$ 
13      Blindly truncate rem to length  $2^{\text{lvl}}$ 
14      reduced[lvl][2j + 1]  $\leftarrow \text{rem}$ 
15 return terms of all leaf remainders
```

Oblivious multi-point evaluation. The multi-point evaluation protocol itself proceeds in two phases. First, a secret-shared subproduct tree is built bottom-up over the padded evaluation points, shown in

Algorithm 4. Each leaf is the linear factor $(x - p_i)$, and internal nodes are products of their children, realized via the NTT-based multiplication above. All polynomials remain secret-shared; padding to the next power of two ensures regular structure and eliminates branching. Note that the length of cur is a public value, and therefore the only if condition in Algorithm 4 is not data-dependent and does not cause any information leakage. Second, Algorithm 5 shows the multi-point evaluation that reduces the input polynomial top-down through the tree. Starting with the full polynomial at the root, each level replaces a parent polynomial by its remainders modulo the two child subproduct polynomials via oblivious division. At the leaves, the constant term of each remainder is exactly the desired evaluation value. All divisions and reductions operate on secret shares; the protocol therefore reveals nothing.

An important implementation detail is the truncation step of Algorithm 5: after computing $\text{rem} \leftarrow \text{PolyDivMod}(\text{cur}, \text{child})$, rem must be truncated to 2^{lvl} coefficients. By the polynomial remainder theorem, $\deg(\text{rem}) < \deg(\text{child}) = 2^{\text{lvl}}$, so all higher-degree coefficients of rem are zero; the truncation merely removes these known zeros. Without this step, cur retains the full length of p at every tree level, keeping each PolyDivMod call at $O(N \log N)$ regardless of depth and yielding $O(N^2 \log N)$ total. This truncation is oblivious because the length 2^{lvl} is determined solely by the publicly known tree structure, not by the secret evaluation points, so no information is revealed.

4.4 Full Protocol

Combining the preprocessing from Algorithm 1 with the Halo2 lookup argument [48], MPlookup proceeds in three phases: trusted setup, collaborative proof generation, and verification.

Let L be the smallest power of two satisfying $L \geq N + \kappa + 1$, where κ denotes the number of blinding rows; let $H = \{1, \omega, \dots, \omega^{L-1}\}$ be the multiplicative subgroup of order L over $\mathbb{F}_p$; and let $L_u = L - \kappa - 1$ denote the number of usable rows. The following fixed public selector polynomials over H are used: $\ell_0(X)$ evaluates to 1 at ω^0 and 0 elsewhere; $q_{\text{blind}}(X)$ evaluates to 1 on the κ blinding rows and 0 elsewhere; $q_{\text{last}}(X)$ evaluates to 1 at ω^{L_u} and 0 elsewhere.

Setup. A trusted initializer runs a universal setup to obtain public parameters pp for a KZG-based polynomial commitment scheme Marlin [10]. A committer key ck and a verifier key vk are derived from pp and distributed to all provers and the verifier. Since the trusted setup is universal [10], the setup procedure can be executed once, shared among multiple zkSNARK applications.

Collaborative Proof Generation. On input secret-shared $\mathbf{f}$ (size M) and public $\mathbf{t}$ (size N), the provers perform the following steps via ABB operations on secret shares. All intermediate polynomial values remain secret-shared throughout; only polynomial commitments, polynomial evaluations, and opening proofs are output as public values. Provers first execute protocol $\Pi_{\text{Preprocess}}$, followed by executing protocol $\Pi_{\text{PermVanish}}$ to get the NIZK proof π . Algorithm 6 in Appendix E.1 shows $\Pi_{\text{PermVanish}}$.

Verification. Given π , vk, and public $\mathbf{t}$:

- (1) Replay $\mathcal{T}$ over the commitments in π to recover β, γ, ζ .
- (2) Verify π_ζ for (f, t, f', t', Z, Q) at ζ ; verify $\pi_{\omega\zeta}$ for Z at $\omega\zeta$; verify $\pi_{\omega^{-1}\zeta}$ for f' at $\omega^{-1}\zeta$.

(3) Compute $Z_H(\zeta) = \zeta^L - 1$ and check $C(\zeta) \stackrel{?}{=} \hat{Q} \cdot Z_H(\zeta)$, where $C(\zeta)$ is evaluated from the five constraint terms using the opened values $\hat{f}, \hat{t}, \hat{f}', \hat{t}', \hat{Z}, \hat{Z}_\omega, \hat{f}'_{\omega^{-1}}$ and the selectors evaluated at ζ .

The verifier accepts if and only if all opening proofs pass and the quotient check holds.

5 Performance Analysis and Security Analysis

We analyze the computational complexity of MPlookup and formally prove its correctness, soundness, obliviousness, and zero-knowledge properties.

5.1 Performance Analysis

We analyze the number of ABB operations required by the full MPlookup protocol.

THEOREM 5.1. *Denoting $N = |t|$, the full MPlookup protocol invokes $O(N \log^2 N)$ ABB operations.*

PROOF. Primitive costs. We first establish the cost of the building blocks used in Algorithm 1.

- *Oblivious Bitonic sort* [3]. Sorting N elements uses $O(N \log^2 N)$ compare-and-swap gates, each invoking $O(1)$ ABB operations.
- *NTT-based polynomial multiplication*. Multiplying two degree- d polynomials with public twiddle factors requires $d \mathcal{F}_{\text{Mult}}$ operations for the pointwise product step; the NTT and inverse-NTT passes use only additions with public constants.
- *Polynomial inversion* (Algorithm 2). Inverting a degree- d polynomial runs for $\lceil \log_2 d \rceil$ iterations; iteration ℓ operates on polynomials of length 2^ℓ at cost $O(2^\ell) \mathcal{F}_{\text{Mult}}$. The total is $\sum_{\ell=1}^{\lceil \log_2 d \rceil} O(2^\ell) = O(d) \mathcal{F}_{\text{Mult}}$.
- *Polynomial division* (Algorithm 3). Dividing a degree- d_{num} numerator by a degree- $d_{\text{den}} \leq d_{\text{num}}$ denominator calls polynomial inversion at degree $d_{\text{num}} - d_{\text{den}}$ plus two polynomial multiplications at degree $O(d_{\text{num}})$, giving $O(d_{\text{num}}) \mathcal{F}_{\text{Mult}}$ in total.

Lookup argument preprocessing costs. Protocol $\Pi_{\text{Preprocess}}$ requires $O(N \log^2 N)$ ABB operations in total. The performance analysis of this protocol is provided below. *Step 1.* Sorting $M \leq N$ secret-shared field elements via Bitonic sort requires $O(N \log^2 N) \mathcal{F}_{\text{LT}}$ and $O(N \log^2 N) \mathcal{F}_{\text{Mult}}$ for the compare-and-swap gates.

Step 2. $N - 1$ equality comparisons requires $O(N) \mathcal{F}_{\text{Eq}}$.

Step 3. Building $P(x) = \prod_{i=1}^M (x - f'_i)$ via a balanced binary tree of NTT multiplications proceeds over $\lceil \log_2 M \rceil$ levels. Level ℓ contains $\lceil M/2^\ell \rceil$ pairs of degree- $2^{\ell-1}$ polynomials, each multiplied at cost $O(2^{\ell-1}) \mathcal{F}_{\text{Mult}}$. Summing over all levels:

$$\sum_{\ell=1}^{\lceil \log_2 M \rceil} \left\lceil \frac{M}{2^\ell} \right\rceil \cdot O(2^{\ell-1}) = O(M \log M) \mathcal{F}_{\text{Mult}}.$$

Step 4. The subproduct tree construction in Algorithm 4 applies the identical level-by-level argument as Step 3, yielding $O(N \log N) \mathcal{F}_{\text{Mult}}$. For the top-down reduction phase in Algorithm 5, at depth d from the root the tree has 2^d nodes, each requiring one polynomial division of a degree- $O(N/2^d)$ parent polynomial by a degree- $O(N/2^d)$ child subproduct, at cost $O(N/2^d) \mathcal{F}_{\text{Mult}}$. Summing over all $\lceil \log_2 N \rceil$ depths:

$$\sum_{d=0}^{\lceil \log_2 N \rceil - 1} 2^d \cdot O\left(\frac{N}{2^d}\right) = O(N \log N) \mathcal{F}_{\text{Mult}}.$$

Step 5. Computing N zero-tests requires $O(N) \mathcal{F}_{\text{IsZero}}$.

Steps 6 and 7. Each sort is on N tuples whose primary sort key is a secret boolean bit, p'_j or $1 - o_i$. Comparing two boolean secrets a and b reduces to evaluating $(1 - a) \cdot b$ via one $\mathcal{F}_{\text{Mult}}$. The secondary sort key is a public index, requiring no additional ABB calls. Each sort therefore costs $O(N \log^2 N) \mathcal{F}_{\text{Mult}}$.

Step 8. The concatenated vector $\mathbf{l}$ has $2N$ elements. The primary sort key is again a secret boolean bit; the secondary key is a secret position index derived from earlier secret-input sorts. Both boolean primary comparison and integer secondary comparison are required, yielding $O(N \log^2 N) \mathcal{F}_{\text{Mult}}$ and $O(N \log^2 N) \mathcal{F}_{\text{LT}}$.

Step 9. Extracting N values from the first N elements of $\mathbf{l}'$ requires $O(N) \mathcal{F}_{\text{Add}}$.

Permutation and vanishing costs. In total, permutation and vanishing arguments protocol $\Pi_{\text{PermVanish}}$ requires $O(N) \mathcal{F}_{\text{Mult}}$, $O(N) \mathcal{F}_{\text{Inverse}}$, and $O(N \log N) \mathcal{F}_{\text{Add}}$. Appendix E.2 shows performance analysis of this protocol.

Summing all steps, MPlookup protocol invokes $O(N \log^2 N)$ ABB operations in total. $\square$

5.2 Security Analysis

We now formally prove that MPlookup satisfies the four security properties stated in Section 3.1. We use λ to denote the security parameter, and $\text{negl}(\lambda)$ to denote a negligible function.

THEOREM 5.2 (COMPLETENESS). *If $\mathbf{f} \subseteq \mathbf{t}$ and all provers are honest,*

$$\Pr[\text{Verify}(\text{vk}, \mathbf{t}, \pi) = 1 \mid \pi \leftarrow \text{Prove}(\text{ck}, \mathbf{f}, \mathbf{t})] = 1.$$

PROOF. By Theorem 4.1, Algorithm 1 on any $\mathbf{f} \subseteq \mathbf{t}$ produces permuted vectors $(\mathbf{f}', \mathbf{t}')$ satisfying Constraints 3.1–3.3 with probability 1. Given correctly formed $(\mathbf{f}', \mathbf{t}')$, the constraint polynomial $C(X)$ vanishes identically on H ; hence $Q(X) = C(X)/Z_H(X)$ is a well-defined polynomial of degree at most $3L$. The grand product closes as $Z(\omega^{L_u}) \in \{0, 1\}$ and satisfies the q_{last} constraint. All KZG commitments and opening proofs are generated correctly by honest provers, so the verifier's quotient check $C(\zeta) = \hat{Q} \cdot Z_H(\zeta)$ holds. $\square$

THEOREM 5.3 (SOUNDNESS). *For all $\mathbf{f} \not\subseteq \mathbf{t}$ and all PPT adversarial provers $\mathcal{A}$,*

$$\Pr[\text{Verify}(\text{vk}, \mathbf{t}, \pi) = 1 \mid \pi \leftarrow \mathcal{A}(\text{ck}, \mathbf{t})] \leq \text{negl}(\lambda).$$

PROOF. Suppose $\mathbf{f} \not\subseteq \mathbf{t}$. Assume $\mathcal{A}$ produces committed polynomials $f(X), f'(X), t'(X), Z(X), Q(X)$ and evaluations that pass verification. The verifier check $C(\zeta) = \hat{Q} \cdot Z_H(\zeta)$ implies that the polynomial $R(X) \triangleq C(X) - Q(X) \cdot Z_H(X)$ evaluates to zero at the random challenge ζ . If $R(X)$ is not identically zero, the Schwartz-Zippel lemma bounds the probability that a uniformly random ζ is a root of R by $\deg(R) / |\mathbb{F}_p| \leq O(L)/p = \text{negl}(\lambda)$. If $R(X) \equiv 0$, then $C(X)$ vanishes on H , meaning all five constraint terms hold at every point of H . In particular, the constraint $(f'(X) - t'(X)) \cdot (f'(X) - f'(\omega^{-1}X)) = 0$ holds on all usable rows, and $t_0(X) \cdot (f'(X) - t'(X)) = 0$ holds at the first row. Together with the grand product Z , these constraints force every element of $\mathbf{f}'$, and hence of $\mathbf{f}$, to appear in $\mathbf{t}$ [48], contradicting $\mathbf{f} \not\subseteq \mathbf{t}$. The grand product relies on random challenges β, γ derived via Fiat-Shamir after commitment; an adversarial forgery succeeding due to a chance collision occurs with probability at most $O(L)/p = \text{negl}(\lambda)$.

Under the KZG binding assumption [10], the adversary cannot open any commitment to two different values. Therefore, $\mathcal{A}$ succeeds with probability at most $\text{negl}(\lambda)$. $\square$

THEOREM 5.4 (OBLIVIOUSNESS). *For any static semi-honest adversary $\mathcal{A}$ corrupting at most t of the n provers, the protocol $\Pi_{\text{Preprocess}}$ computationally realizes the ideal functionality $\mathcal{F}_{\text{Preprocess}}$ in the $\mathcal{F}_{\text{ABB}}$ -hybrid model. In particular, for any two input pairs $(\mathbf{f}_0, \mathbf{t})$ and $(\mathbf{f}_1, \mathbf{t})$ with $|\mathbf{f}_0| = |\mathbf{f}_1|$ and $\mathbf{f}_0, \mathbf{f}_1 \subseteq \mathbf{t}$,*

$$\text{View}_{\mathcal{A}}^{\Pi_{\text{Preprocess}}}(\mathbf{f}_0, \mathbf{t}) \stackrel{c}{\approx} \text{View}_{\mathcal{A}}^{\Pi_{\text{Preprocess}}}(\mathbf{f}_1, \mathbf{t}),$$

where $\stackrel{c}{\approx}$ denotes computational indistinguishability.

PROOF. We show that Algorithm 1 contains no data-dependent leakage to any t -party collusion, by classifying every operation:

(1) *ABB operations $\mathcal{F}_{\text{Mult}}$, $\mathcal{F}_{\text{Eq}}$, $\mathcal{F}_{\text{LT}}$, $\mathcal{F}_{\text{IsZero}}$, and $\mathcal{F}_{\text{Add}}$.* By the ideal-world security of $\mathcal{F}_{\text{ABB}}$, any collusion of at most t corrupted provers learns nothing about honest provers' private inputs beyond what is already implicit in their own shares.

(2) *Arithmetic on public data.* The evaluation points $t_1, \dots, t_N$ in Step 4 and all position indices used in Steps 6–8 are public values; operations on them introduce no new secret.

(3) *Oblivious sorts.* Each sort is realized as a fixed, input-independent Bitonic-sort [3] network of ABB compare-and-swap gates. The network topology depends only on the vector length, which is a public parameter; hence the sequence of ABB calls is the same for all inputs of equal size.

(4) *Control flow.* All loop bounds and branching in Algorithms 2–5 depend only on publicly known polynomial degrees. There is no secret-dependent branching anywhere in the protocol.

We construct a simulator Sim that receives the input/output shares of the corrupted parties and the public parameters $M, \mathbf{t}$. For every ABB invocation, Sim queries $\mathcal{F}_{\text{ABB}}$ and returns fresh shares indistinguishable from real protocol shares. Since all sort networks and polynomial-degree parameters are public, the sequence of $\mathcal{F}_{\text{ABB}}$ calls is identical for $\mathbf{f}_0$ and $\mathbf{f}_1$. By the UC composition theorem [33], the composed execution of $\Pi_{\text{Preprocess}}$ in the $\mathcal{F}_{\text{ABB}}$ -hybrid model is computationally indistinguishable from the ideal-world execution under $\mathcal{F}_{\text{Preprocess}}$, establishing obliviousness. $\square$

THEOREM 5.5 (OBLIVIOUSNESS OF THE PERMUTATION AND VANISHING ARGUMENTS). *For any static semi-honest adversary $\mathcal{A}$ corrupting at most t of the n provers, the protocol $\Pi_{\text{PermVanish}}$ computationally realizes the ideal functionality $\mathcal{F}_{\text{PermVanish}}$ in the $\mathcal{F}_{\text{ABB}}$ -hybrid model. In particular, for any two secret-shared input pairs $(\mathbf{f}'_0, \mathbf{t}'_0)$ and $(\mathbf{f}'_1, \mathbf{t}'_1)$ of the same size satisfying the alignment constraints,*

$$\text{View}_{\mathcal{A}}^{\Pi_{\text{PermVanish}}}(\mathbf{f}'_0, \mathbf{t}'_0) \stackrel{c}{\approx} \text{View}_{\mathcal{A}}^{\Pi_{\text{PermVanish}}}(\mathbf{f}'_1, \mathbf{t}'_1).$$

The proof of Theorem 5.5 is provided in Appendix E.3.

COROLLARY 5.6 (OBLIVIOUSNESS OF MPLOOKUP). *The full MPlookup protocol Π_{MPlookup} is oblivious: for any static semi-honest adversary $\mathcal{A}$ corrupting at most t provers, and any two valid input pairs $(\mathbf{f}_0, \mathbf{t})$ and $(\mathbf{f}_1, \mathbf{t})$ with $|\mathbf{f}_0| = |\mathbf{f}_1|$ and $\mathbf{f}_0, \mathbf{f}_1 \subseteq \mathbf{t}$,*

$$\text{View}_{\mathcal{A}}^{\Pi_{\text{MPlookup}}}(\mathbf{f}_0, \mathbf{t}) \stackrel{c}{\approx} \text{View}_{\mathcal{A}}^{\Pi_{\text{MPlookup}}}(\mathbf{f}_1, \mathbf{t}).$$

PROOF. Π_{MPlookup} composes $\Pi_{\text{Preprocess}}$ and $\Pi_{\text{PermVanish}}$. The output $(\mathbf{f}', \mathbf{t}')$ of preprocessing is passed as secret-shared input to $\Pi_{\text{PermVanish}}$ without any intermediate opening. By the UC composition theorem, the sequential composition of two protocols that each UC-realize their respective ideal functionalities in the $\mathcal{F}_{\text{ABB}}$ -hybrid model also UC-realizes the composed ideal functionality, completing the obliviousness proof for the full protocol. $\square$

THEOREM 5.7 (ZERO-KNOWLEDGE). *There exists a PPT simulator Sim such that for all $\mathbf{f} \subseteq \mathbf{t}$, all provers $P_1, \dots, P_n$, and all efficient distinguishers D ,*

$$|\Pr[D(\pi) = 1] - \Pr[D(\tilde{\pi}) = 1 \mid \tilde{\pi} \leftarrow \text{Sim}(\text{vk}, \mathbf{t})]| \leq \text{negl}(\lambda),$$

where π is a proof produced by honest provers on $(\mathbf{f}, \mathbf{t})$.

PROOF. The proof π consists exclusively of public values: six KZG commitments, eight field-element evaluations, and three KZG opening proofs, as described in Section 4.4. None of these elements directly encodes the secret query vector $\mathbf{f}$.

We construct $\text{Sim}(\text{vk}, \mathbf{t})$ as follows. Since the simulator operates in the random oracle model, it can program the random oracle.

(1) Program the Fiat-Shamir random oracle to produce challenges β, γ, ζ consistently with all simulated commitments.

(2) For $f(X)$: choose L_u uniformly random field elements as the evaluations on usable rows, and set the final $\kappa + 1$ entries to the blinding and selector values. Interpolate via INTT to obtain polynomial coefficients and compute the KZG commitment f_{Com} .

(3) For $f'(X)$: apply the same procedure to obtain simulated $f'(X)$ and f'_{Com} .

(4) For $t(X)$ and $t'(X)$: commit to the public table vectors as in the real protocol; these are deterministic functions of $\mathbf{t}$.

(5) Derive $Z(X)$ satisfying $Z(\omega^0) = 1$ under the simulated challenges β, γ . Compute $Q(X) = C(X)/Z_H(X)$ from all simulated polynomials and commit.

(6) At the simulated challenge ζ , evaluate all polynomials and produce KZG opening proofs for the simulated commitments.

By the KZG commitment hiding property [10], the commitments to $f(X)$ and $f'(X)$ are computationally indistinguishable from commitments to any polynomial sharing the same blinding values. The $\kappa \geq 1$ blinding rows ensure that polynomial evaluations at the random challenge ζ are uniformly distributed over $\mathbb{F}_p$ from the verifier's perspective [48]; hence simulated evaluations $\hat{f}, \hat{f}'$ are indistinguishable from real ones. By Corollary 5.6, both $\Pi_{\text{Preprocess}}$ and $\Pi_{\text{PermVanish}}$ reveal nothing about $\mathbf{f}$ to any party during proof generation. Combining these, the full simulated transcript $\tilde{\pi}$ is computationally indistinguishable from a real proof π , so the verifier learns nothing about $\mathbf{f}$ beyond $\mathbf{f} \subseteq \mathbf{t}$. $\square$

6 Implementation and Evaluation

This section describes our implementation of MPlookup and presents experimental results addressing scalability, step-level performance breakdown, and multi-party scaling.

6.1 Implementation

We implement MPlookup as a full open-source Rust library, built upon the collaborative zkSNARKs framework of Ozdemir et al. [33]. The arithmetic black box is realized using CompatCircuit [37],

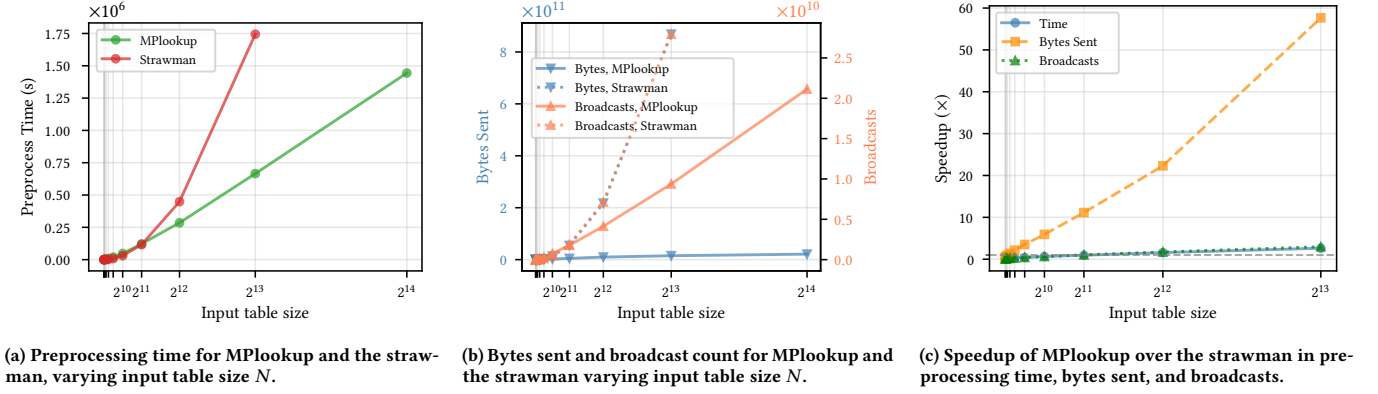


Figure 7: Evaluation of RQ1: scalability and comparison with the strawman. Input query vector size $M = \frac{N}{2}$, 2 parties.

which provides a semi-honest MPC protocol over the BLS12-377 scalar field $\mathbb{F}_p$. The library is organized into four modules.

- `halo2_lookup` implements the collaborative permutation and vanishing arguments ($\Pi_{\text{PermVanish}}$, Algorithm 6) using a Marlin-style KZG polynomial commitment scheme [10] over BLS12-377.
- `permutation` implements the full MPlookup preprocessing protocol (Algorithm 1) and the $O(N^2)$ strawman baseline (Algorithm 7).
- `polynomial` provides NTT-based polynomial multiplication using the Arkworks `ark-poly` [33] library, Newton-iteration polynomial inversion (Algorithm 2), division with remainder (Algorithm 3), subproduct tree construction (Algorithm 4), and multi-point evaluation (Algorithm 5).
- `sorting` implements oblivious bitonic sort [3] over MPC field elements via secure compare-and-swap gates that invoke the ABB's $\mathcal{F}_{\text{LT}}$ primitive.

The network layer uses `mpc-net` [33] with a king-based star topology for inter-party communication. Each party holds additive shares of the secret query vector $\mathbf{f}$; the public table $\mathbf{t}$ is available in plaintext to all parties. Our current implementation is single-threaded per party, with additional CPU cores used solely to handle communication with other parties.

6.2 Experimental Setup

Our experiments were conducted on a server instance on the ESXi 8.0 virtualization platform. Denote the prover count as n . The server instance simulating n provers was allocated with up to $4n$ vCPU cores from Intel Xeon 4214R @ 2.4 GHz, 16n GB of RAM, running Ubuntu Linux 24.04 LTS. We set up n provers in a local network. Note that our implementation is *not* parallelized, but more than one CPU core is needed to handle the communication.

We focus our evaluation on the preprocessing phase (Algorithm 1) and the strawman baseline (Algorithm 7), as this phase dominates end-to-end proof generation time. We also execute $\Pi_{\text{PermVanish}}$ and report its timing for completeness. Unless stated otherwise, we use $n = 2$ parties and set the query size to $M = N/2$, where N is the table size. For the scalability experiment we vary $N \in \{8, 16, 32, \dots, 16384\}$ for MPlookup and $N \in \{8, 16, 32, \dots, 8192\}$

for the strawman. For the multi-party experiment we fix $N = 1,024$ and vary $n \in \{2, 4, 8, 16\}$.

We investigate the following research questions.

- **RQ1:** Does MPlookup have $O(N \log^2 N)$ complexity, and at what scale does it outperform the $O(N^2)$ strawman in both time and communication?
- **RQ2:** Which algorithmic steps dominate the running time, and does the observed result match the theoretical analysis?
- **RQ3:** How does MPlookup perform as the number of participating parties increases?

We fully adhere to the open science policy. *All numerical data in this evaluation section are directly computed using our script and are verifiable. The script will be provided in Open Science section.*

6.3 RQ1: Scalability and Comparison

Figures 7a and 7b plot preprocessing time, bytes sent, and broadcast count. We confirm MPlookup scales as $O(N \log^2 N)$ and the strawman as $O(N^2)$, in agreement with Theorem 5.1.

At small N , the strawman is faster due to lower constant factors: each iteration uses one $\mathcal{F}_{\text{Eq}}$ call, whereas MPlookup invokes sorting networks and polynomial arithmetic. The crossover lies between $N = 2,048$ and $N = 4,096$. At $N = 8,192$, MPlookup achieves a 2.62 $\times$ speedup, completing in 665,448 s vs. 1,744,437 s for the strawman. From $N = 2,048$ to 16,384, the observed doubling ratios 2.34 $\times$, 2.33 $\times$, 2.17 $\times$ closely match the theoretical 2.38 $\times$, 2.35 $\times$, 2.32 $\times$.

Figure 7c shows speedups in preprocessing time, bytes sent, and broadcast count; the time speedup turns decisively in MPlookup's favour beyond $N = 4,096$. Communication savings are more pronounced: at $N = 8,192$, MPlookup sends approximately 58 $\times$ fewer bytes, as the strawman's $O(N^2)$ equality comparisons each require a full MPC round. Broadcast count is higher for MPlookup at small N due to sorting and subproduct tree rounds; at $N = 2,048$, MPlookup also issues fewer broadcasts.

6.4 RQ2: Step-Level Performance Breakdown

Figure 8a shows the time cost contributed by each of the nine steps of Algorithm 1 across all tested table sizes. Figure 8b plots the time cost of each step divided by $N \log^2 N$ for Steps 1, 4, 6,

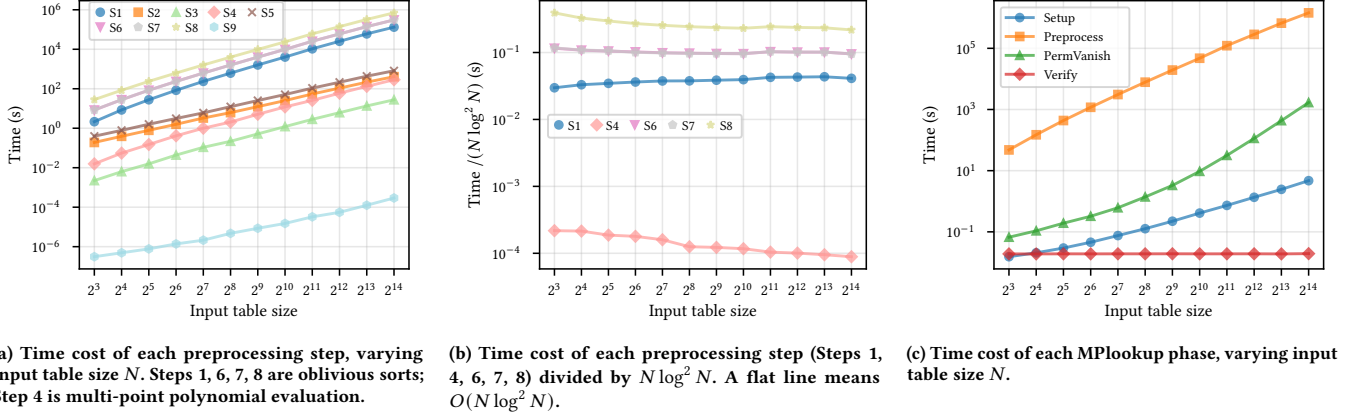


Figure 8: Evaluation of RQ2: step-level performance breakdown. Input query vector size $M = \frac{N}{2}$, 2 parties.

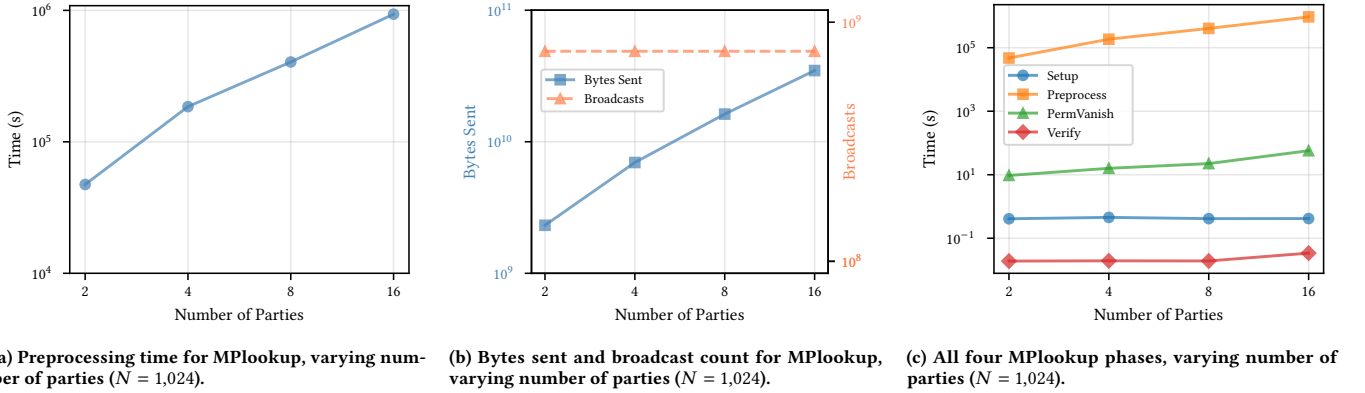


Figure 9: Evaluation of RQ3: scaling with number of parties. Input table vector size $N = 1,024$, query vector size $M = 512$.

7, 8. For Steps 1, 6, 7, 8, flat curves confirm $O(N \log^2 N)$ scaling. For Step 4, the line is close to $O(N \log N)$ scaling. This is because both subproduct tree construction ($O(N \log N)$) and multi-point evaluation ($O(N \log^2 N)$) are included in Step 4, and the subproduct tree construction contributes more time cost.

Figure 8c breaks down the four phases of MPlookup: setup, proof generation consisting of $\Pi_{\text{Preprocess}}$ and $\Pi_{\text{PermVanish}}$, and finally verification. $\Pi_{\text{Preprocess}}$ overwhelmingly dominates the total cost across all tested sizes, while verification remains essentially constant at approximately 19 ms regardless of N . $\Pi_{\text{PermVanish}}$ and setup both grow with N but contribute less than 0.2% of total time, confirming that preprocessing is the decisive bottleneck.

6.5 RQ3: Scaling with the Number of Parties

Figure 9a shows preprocessing time at $N = 1,024$, $M = 512$: from 47,389 s for 2 parties to 185,312 s, 404,915 s, and 935,427 s—slowdowns of 3.9 $\times$, 8.5 $\times$, and 19.7 $\times$ over the 2-party baseline. Figure 9b shows bytes sent scales similarly to time, while broadcast count remains constant across party counts, consistent with the communication structure.

In Figure 9c, the setup time and verification time remain stable because they are single-user protocols, while $\Pi_{\text{PermVanish}}$ grows from 9.5 s at 2 parties to 57.1 s at 16 parties as additional parties require more collaborative KZG commitment rounds.

We note that the evaluation results are measured with ComPatCircuit as the ABB, whose $\mathcal{F}_{\text{LT}}$ operation requires $O(\log_2 p)$ sequential communication rounds per comparison over the BLS12-377 scalar field, where $\log_2 p \approx 253$. This is the primary source of the large constant factor observed throughout.

7 Discussion

We discuss two directions for future improvement of MPlookup: reducing communication overhead through a more efficient Less-Than protocol, and extending the construction to the malicious security model.

7.0.1 Further Communication Round and Time Cost Optimization. Our MPlookup protocol is comparison-intensive. In our implementation, we integrated ComPatCircuit [37] to implement the ABB, which has a $\log_2 p$ rounds of communication in Less-Than operation.

To further optimize performance, recently, Wang et al. [44] proposed a dealer-assisted n -party Less-Than-Bits extraction protocol in $\mathbb{F}_p$ and it achieves constant-round online complexity. Considering that our current evaluation result is already able to demonstrate the $O(N \log^2 N)$ complexity of MPlookup protocol, we leave further optimizations as future work.

7.0.2 Maliciously Secure Protocol. Our MPlookup implementation is semi-honest. This is based on the consideration where the typical usage of a multi-party lookup argument is to generate a public NIZK proof *after finishing the computation*, and even if MPlookup output is corrupted by a malicious adversary, the NIZK proof verification would fail, guaranteed by the soundness property of the underlying Halo2 lookup argument protocol [48]. For a stricter scenario where the adversary may choose to corrupt computations for information leakage, MPlookup can also be made maliciously secure by using a maliciously secure MPC protocol to implement the ABB and adapt a maliciously secure collaborative zkSNARK [22], left as future work.

8 Conclusion

We have presented MPlookup, the first multi-party lookup argument for collaborative zkSNARKs, achieving quasilinear $O(N \log^2 N)$ complexity through four oblivious sorting operations and a novel oblivious multi-point evaluation protocol. We have formally proved that MPlookup satisfies obliviousness, completeness, soundness, and zero-knowledge, and provided an open-source Rust implementation. We have confirmed the $O(N \log^2 N)$ complexity through our evaluation and discussed possible future work.

Acknowledgments

Claude was used for minor editorial improvements and grammar checks. GitHub Copilot assisted in the development of the implementation. All outputs were manually verified by the authors.

References

- [1] Mohammed Alghazwi, Tariq Bontekoe, Leon Visscher, and Fatih Turkmen. 2024. Collaborative CP-NIZKs: Modular, Composable Proofs for Distributed Secrets. *arXiv preprint arXiv:2407.19212* (2024).
- [2] Arasu Arun, Srinath Setty, and Justin Thaler. 2024. Jolt: Snarks for virtual machines via lookups. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 3–33.
- [3] Kenneth E Batcher. 1968. Sorting networks and their applications. In *Proceedings of the April 30–May 2, 1968, spring joint computer conference*. 307–314.
- [4] Olivier Bégassat, Alexandre Belling, Théodore Chapuis-Chkaiban, and Nicolas Liochon. 2021. A specification for a ZK-EVM. *Ethereum Res* (2021).
- [5] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. 2019. *Completeness theorems for non-cryptographic fault-tolerant distributed computation*. Association for Computing Machinery, New York, NY, USA, 351–371. <https://doi.org/10.1145/3335741.3335756>
- [6] Rishabh Bhadauria, Carmit Hazay, Muthuramakrishnan Venkatasubramanian, Wenxuan Wu, and Yupeng Zhang. 2023. Private polynomial commitments and applications to MPC. In *IACR International Conference on Public-Key Cryptography*. Springer, 127–158.
- [7] Vishwas Bhargava, Sumanta Ghosh, Zeyu Guo, Mrinal Kumar, and Chris Umans. 2024. Fast multivariate multipoint evaluation over all finite fields. *J. ACM* 71, 3 (2024), 1–32.
- [8] Matteo Campanelli, Antonio Faonio, Dario Fiore, Tianyu Li, and Helger Lipmaa. 2024. Lookup arguments: improvements, extensions and applications to zero-knowledge decision trees. In *IACR International Conference on Public-Key Cryptography*. Springer, 337–369.
- [9] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. 2023. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 499–530.
- [10] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Noah Vesely, and Nicholas Ward. 2020. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In *Advances in Cryptology—EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I* 39. Springer, 738–768. https://link.springer.com/chapter/10.1007/978-3-030-45721-1_26.
- [11] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. 2012. Multiparty computation from somewhat homomorphic encryption. In *Annual Cryptology Conference*. Springer, 643–662. https://link.springer.com/chapter/10.1007/978-3-642-32009-5_38.
- [12] Neil Daswani and Moudy Elbayadi. 2021. *Big breaches: Cybersecurity lessons for everyone*. Springer.
- [13] Minglang Dong, Cong Zhang, Yujie Bai, and Yu Chen. 2025. Efficient {Multi-Party} Private Set Union Without {Non-Collusion} Assumptions. In *34th USENIX Security Symposium (USENIX Security 25)*. 2005–2024.
- [14] Moumita Dutta, Chaya Ganesh, Sikhar Patranabis, Shubh Prakash, and Nitin Singh. 2024. Batching-efficient RAM using updatable lookup arguments. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 4077–4091.
- [15] Melanie Evans, Anna Wilde Mathews, and Tom McGinty. 2021. Hospitals often charge uninsured people the highest prices, new data show. *Wall Street Journal* (2021).
- [16] Zhiyong Fang, Sanjam Garg, Bhaskar Roberts, Wenxuan Wu, and Yupeng Zhang. 2025. *Collaborative zkSNARKs with sublinear prover time and constant proof size*. Cryptology ePrint Archive. <https://eprint.iacr.org/2025/1388>
- [17] Keith Frikken. 2007. Privacy-preserving set union. In *International Conference on Applied Cryptography and Network Security*. Springer, 237–252.
- [18] Ariel Gabizon and Dmitry Khovratovich. 2022. flookup: Fractional decomposition-based lookups in quasi-linear time independent of table size. *Cryptology ePrint Archive* (2022).
- [19] Ariel Gabizon and Zachary J Williamson. 2020. plookup: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive* (2020). <https://eprint.iacr.org/2020/315>.
- [20] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. 2019. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* (2019). <https://eprint.iacr.org/2019/953>.
- [21] Sinka Gao, Guoqiang Li, and Hongfei Fu. 2024. ZKWASM: A ZKSNARK WASM Emulator. *IEEE Transactions on Services Computing* (2024). <https://ieeexplore.ieee.org/abstract/document/10587123>.
- [22] Sanjam Garg, Aarushi Goel, Abhishek Jain, Bhaskar Roberts, and Sruthi Sekar. 2025. Malicious security in collaborative zk-SNARKs: more than meets the eye. In *Annual International Cryptology Conference*. Springer, 396–428.
- [23] Sumanta Ghosh, Prahladh Harsha, Simao Herdade, Mrinal Kumar, and Ramprasad Saptharishi. 2023. Fast numerical multivariate multipoint evaluation. In *2023 IEEE 64th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1426–1439.
- [24] Pascal Giorgi, Fabien Laguillaumie, Lucas Ottow, and Damien Vergnaud. 2024. Fast secure computations on shared polynomials and applications to private set operations. In *5th Conference on Information-Theoretic Cryptography (ITC 2024)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 11–1.
- [25] Jens Groth. 2016. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology—EUROCRYPT 2016: 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Vienna, Austria, May 8–12, 2016, Proceedings, Part II* 35. Springer, 305–326. https://link.springer.com/chapter/10.1007/978-3-662-49896-5_11.
- [26] Hossein Hafezi, Gaspard Anthoine, Matteo Campanelli, and Dario Fiore. 2025. SoK: Lookup Table Arguments. *Cryptology ePrint Archive* (2025).
- [27] Marcel Keller. 2020. MP-SPDZ: A versatile framework for multi-party computation. In *Proceedings of the 2020 ACM SIGSAC conference on computer and communications security*. 1575–1590. <https://dl.acm.org/doi/abs/10.1145/3372297.3417872>.
- [28] Alexander Kobel and Michael Sagraloff. 2013. Fast approximate polynomial multipoint evaluation and applications. *arXiv preprint arXiv:1304.8069* (2013).
- [29] Polygon Labs. 2024. Polygon Miden | A rollout for high-throughput, private applications. <https://polygon.technology/polygon-miden> Accessed: April 2, 2026.
- [30] Tianyi Liu, Zhenfei Zhang, Yuncong Zhang, Wenqing Hu, and Ye Zhang. 2024. Ceno: Non-uniform, Segment and Parallel Zero-knowledge Virtual Machine. *Cryptology ePrint Archive* (2024). <https://eprint.iacr.org/2024/387>.
- [31] Xuanming Liu, Zhelei Zhou, Yinghao Wang, Yanxin Pang, Jinye He, Bingsheng Zhang, Xiaohu Yang, and Jiaheng Zhang. 2025. Scalable collaborative zk-SNARK and its application to fully distributed proof delegation. In *34th USENIX Security Symposium (USENIX Security 25)*.
- [32] Mary Maller, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. 2019. Sonic: Zero-knowledge SNARKs from linear-size universal and updatable structured reference strings. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2111–2128. <https://dl.acm.org/doi/abs/10.1145/3319535.3339817>.

- [33] Alex Ozdemir and Dan Boneh. 2022. Experimenting with collaborative zk-SNARKs: Zero-Knowledge proofs for distributed secrets. In *31st USENIX Security Symposium (USENIX Security 22)*. 4291–4308. <https://www.usenix.org/conference/usenixsecurity22/presentation/ozdemir>.
- [34] Shahar Papini and Ulrich Haböck. 2023. Improving logarithmic derivative lookups using GKR. *Cryptology ePrint Archive* (2023).
- [35] Luke Pearson, Joshua Fitzgerald, Héctor Masip Ardévol, Marta Ballés Muñoz, and José Luis Muñoz Tapia. 2022. PlonkUp: Reconciling plonk with plookup. (2022).
- [36] Jim Posen and Assimakis A Kattis. 2022. Caulk+: Table-independent lookup arguments. *Cryptology ePrint Archive* (2022).
- [37] Huayi Qi, Minghui Xu, Xiaohua Jia, and Xiuzhen Cheng. 2026. VDORAM: Towards a random access machine with both public verifiability and distributed obliviousness. In *Network and Distributed System Security Symposium (NDSS)*.
- [38] Alex Sangers, Maran van Heesch, Thomas Attema, Thijs Veugen, Mark Wiggerman, Jan Veldsink, Oscar Bloemen, and Daniël Worm. 2019. Secure multiparty PageRank algorithm for collaborative fraud detection. In *International Conference on Financial Cryptography and Data Security*. Springer, 605–623.
- [39] Srinath Setty, Justin Thaler, and Riad Wahby. 2024. Unlocking the lookup singularity with lasso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 180–209.
- [40] Alan Szeplieniec, Alexander Lemmens, Jan Ferdinand Sauer, Bobbin Threadbare, et al. 2023. The Tip5 hash function for recursive STARKs. *Cryptology ePrint Archive* (2023).
- [41] Dien HA Tran, Tam NB Nguyen, Nhien-An Le-Khac, and Thuc D Nguyen. 2025. LogaLookup: Efficient Multivariate Lookup Argument for Accelerated Proof Generation. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*. 215–230.
- [42] Joris van der Hoeven and Grégoire Lecerc. 2025. Faster multi-point evaluation over any field. *Applicable Algebra in Engineering, Communication and Computing* (2025), 1–32.
- [43] Triton VM. [n. d.]. Triton VM. <https://triton-vm.org/>. Accessed: April 2, 2026.
- [44] Kaiwen Wang, Xiaolin Chang, Yuehan Dong, and Ruichen Zhang. 2026. Efficient Multi-Party Secure Comparison over Different Domains with Preprocessing Assistance. *arXiv preprint arXiv:2602.19604* (2026).
- [45] Yunbo Yang, Yuejia Cheng, Haibo Tang, Guomin Yang, Bingsheng Zhang, and Kui Ren. 2026. SoK: Understanding zkVM: From Research to Practice. *Cryptology ePrint Archive* (2026).
- [46] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. 2022. Caulk: Lookup arguments in sublinear time. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 3121–3134.
- [47] Arantxa Zapico, Ariel Gabizon, Dmitry Khovratovich, Mary Maller, and Carla Ràfols. 2022. Baloo: Nearly optimal lookup arguments. *Cryptology ePrint Archive* (2022).
- [48] Zcash. [n. d.]. The halo2 book. <https://zcash.github.io/halo2/index.html>. Accessed: April 2, 2026.
- [49] RISC Zero. [n. d.]. RISC Zero | Universal Zero Knowledge. <https://www.risczero.com/>. Accessed: April 2, 2026.
- [50] Yuncong Zhang, Shi-Feng Sun, and Dawu Gu. 2024. Efficient KZG-based univariate sum-check and lookup argument. In *IACR International Conference on Public-Key Cryptography*. Springer, 400–425.
- [51] Yuncong Zhang, Shi-Feng Sun, Ren Zhang, and Dawu Gu. 2023. Polynomial IOPs for memory consistency checks in zero-knowledge virtual machines. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 111–141.

A Open Science

We fully adhere to the open science policy. Our artifacts are available at <https://github.com/MPlookup/mplookup-artifacts>, including:

- Our implementation of $O(N \log^2 N)$ MPlookup. It also includes the $O(N^2)$ strawman for comparison.
- A simplified, easy-to-read single-party emulation of the MPlookup preprocessing algorithm.
- A script to run the experiments.
- Collected experiment results.
- A script to draw the figures in the Evaluation section.
- A script to produce all the numerical data used in the Evaluation section.

Appendix G includes instructions to run the experiments and analyze results.

B Lookup Argument Usages in zkVMs

This appendix surveys how lookup arguments are used in existing zero-knowledge virtual machines, motivating the need for a multi-party lookup argument.

Table 2: Lookup argument schemes and their usages in zero-knowledge virtual machines.

zkVM scheme	Lookup scheme	Usage
Jolt [2]	Lasso [39]	Lookup singularity.
Bégassat et al. [4]	Plookup [19]	Submodule inclusion.
ZKWASM [21]	Plookup [19]	(1) Memory consistency check; (2) Calling frame table.
Zhang et al. [51]	gcq [51]	Memory consistency check. (1) Non-linear operations;
Ceno [30]	logUp [34]	(2) Bytecode retrieval verification; (3) Range check.
Dutta et al. [14]	Updatable lookup [14]	Extract smaller RAMs.
Triton VM [43]	Tip5 [40]	(1) Memory consistency check; (2) Table linking.
VDORAM [37] (Oblivious)	-	No lookup usage due to the absence of a multi-party lookup argument.

Table 2 summarizes lookup argument usages in recent zero-knowledge virtual machines. Lookup arguments are widely used in single-prover zkVMs, while the only multi-prover zkVM, VDORAM [37], does not integrate a lookup argument and suffers from a performance loss.

C Notation Summary

Table 3 summarizes the symbols used throughout the paper.

D Correctness Proof of Theorem 4.1

PROOF. Let $M = |\mathbf{f}|$ and let d denote the number of *distinct* values in $\mathbf{f}$. Since $\mathbf{f} \subseteq \mathbf{t}$ and $\mathbf{t}$ has unique elements, d is also the count of distinct table entries that are queried.

Proof of Constraint 3.1. Step 1 produces $\mathbf{f}'$ by sorting $\mathbf{f}$, which is by definition a permutation. Therefore, Constraint 3.1, that $\mathbf{f}'$ is a permutation of $\mathbf{f}$, holds.

Proof of Constraint 3.2. We first establish two counting facts.

FACT D.1. *In the indicator vector $\mathbf{o}$ produced by Step 2, exactly d entries equal 1, marking the d first-occurrence positions in $\mathbf{f}'$; the remaining $N - d$ entries equal 0, covering both non-first-occurrence positions and the $N - M$ padded positions appended in Step 1.*

Because $\mathbf{f}'$ is sorted, identical values are adjacent. Hence $o_i = 1$ if and only if $i \leq M$ and f'_i differs from its predecessor, where $o_1 = 1$ by definition. Exactly d indices in $[1, N]$ satisfy $o_i = 1$; the remaining $N - d$ indices satisfy $o_i = 0$, a count that includes all $N - M$ padded positions set to $o_i = 0$ by Step 2.

Table 3: Summary of Notation

Symbol	Description
n	Number of MPC provers
t	Maximum corrupted provers
$\mathbf{t}$	Public table vector
$\mathbf{f}$	Secret-shared query vector
$\mathbf{t}'$	Permuted secret-shared table vector
$\mathbf{f}'$	Permuted secret-shared query vector
N	Length of table vectors $\mathbf{t}$ and $\mathbf{t}'$
M	Length of query vectors $\mathbf{f}$ and $\mathbf{f}'$ ($M \leq N$)
λ	Security parameter
p	Prime characteristic of $\mathbb{F}_p$
$\mathbb{F}_p$	Prime field
L	Evaluation domain size (smallest power of 2: $L \geq N + \kappa + 1$)
L_u	Number of usable rows ($L_u = L - \kappa - 1$)
κ	Number of KZG blinding rows
H	Multiplicative subgroup of order L over $\mathbb{F}_p$
ω	Primitive L -th root of unity (generator of H)
pp	Public parameters from trusted setup
ck	Committer key
vk	Verifier key
$Z_H(X)$	Vanishing polynomial $X^L - 1$ over H
β, γ	Fiat-Shamir challenges for the grand product
ζ	Fiat-Shamir evaluation challenge
$Z(X)$	Grand product polynomial
$Q(X)$	Quotient polynomial ($C(X)/Z_H(X)$)
$C(X)$	Combined constraint polynomial

FACT D.2. *In the boolean vector $\mathbf{p}'$ produced by Steps 3 through 5, exactly d entries equal 0, one for each table element whose value appears in $\mathbf{f}$; the remaining $N - d$ entries equal 1, one for each table element whose value does not appear in $\mathbf{f}$.*

The polynomial $P(x) = \prod_{i=1}^M (x - f'_i)$ has d distinct query values as its roots, and the multiplicities sum to M . Since $\mathbf{t}$ contains unique elements, $P(t_j) = 0$ if and only if t_j is one of those d distinct roots, i.e., $t_j \in \mathbf{f}$. After normalization in Step 5, $p'_j = 1$ iff $t_j \notin \mathbf{f}$ and $p'_j = 0$ iff $t_j \in \mathbf{f}$. Hence exactly d entries satisfy $p'_j = 0$; these are the *used* table elements. The remaining $N - d$ entries satisfy $p'_j = 1$; these are the *unused* table elements.

After the sort in Step 6, $\mathbf{r}'$ lists the $N - d$ unused table-element triples first, followed by the d used triples. After the sort in Step 7, $\mathbf{s}'$ lists the $N - d$ non-first-occurrence positions, each with validity bit $1 - o_i = 1$, first, followed by the d first-occurrence positions, each with validity bit 0.

CLAIM D.1. *For each position $k \in [1, N]$, exactly one triple in $\mathbf{I} = \mathbf{I}^{(1)} \parallel \mathbf{I}^{(2)}$ has first component equal to k and third component equal to 1.*

- If $o_k = 1$, meaning k is a first-occurrence position: $\mathbf{I}^{(2)}$ contributes $(k, f'_k, 1)$. Because $o_k = 1$, position k appears among the last d entries of $\mathbf{s}'$ where the validity bit is 0, so every $\mathbf{I}^{(1)}$ triple with first component k carries third component 0. Thus the only valid triple for position k comes from $\mathbf{I}^{(2)}$.

- If $o_k = 0$, meaning k is a non-first-occurrence or padded position: $\mathbf{I}^{(2)}$ contributes $(k, f''_k, 0)$, which is not valid. Because

$o_k = 0$, position k appears among the first $N - d$ entries of $\mathbf{s}'$ where the validity bit is 1. The corresponding $\mathbf{I}^{(1)}$ entry is $(k, v, 1)$ where $v = r'_i[2]$ is an unused table element. Thus the only valid triple for position k comes from $\mathbf{I}^{(1)}$.

Since the N valid triples in $\mathbf{I}$ cover each position in $[1, N]$ exactly once, the sort in Step 8 moves these triples to the first N positions of $\mathbf{I}'$, ordered by position index. Extracting the second components gives $\mathbf{t}'$ whose values are:

- For each of the d first-occurrence positions k : value f'_k , which equals the unique table element t_j satisfying $t_j = f'_k$; this holds since $\mathbf{f} \subseteq \mathbf{t}$ and $\mathbf{t}$ has unique elements.

- For each of the $N - d$ non-first-occurrence positions: one of the $N - d$ unused table elements, assigned one-to-one by the aligned ordering of $\mathbf{r}'$ and $\mathbf{s}'$.

Together these comprise every element of $\mathbf{t}$ exactly once, so $\mathbf{t}'$ is a permutation of $\mathbf{t}$.

Proof of Constraint 3.3. By Claim D.1, for each first-occurrence position i where $o_i = 1$, the unique valid triple with first component i is $(i, f'_i, 1)$ from $\mathbf{I}^{(2)}$. After the sort in Step 8, this triple occupies position i in the first N entries of $\mathbf{I}'$, yielding $t'_i = \mathbf{I}'_i[2] = f'_i$. $\square$

E Permutation and Vanishing Arguments

This appendix presents the full $\Pi_{\text{PermVanish}}$ protocol, along with its performance analysis and the proof of Theorem 5.5.

E.1 The Protocol

Algorithm 6 shows the MPlookup permutation and vanishing arguments protocol $\Pi_{\text{PermVanish}}$. Together, $\Pi_{\text{Preprocess}}$ and $\Pi_{\text{PermVanish}}$ constitute the full MPlookup proof generation process: preprocessing produces the secret-shared aligned pair $(\mathbf{f}', \mathbf{t}')$, and $\Pi_{\text{PermVanish}}$ transforms this into a public NIZK proof π that the verifier can check without any secret data.

The protocol proceeds in five steps: it first encodes the four input vectors $\mathbf{f}, \mathbf{t}, \mathbf{f}', \mathbf{t}'$ as polynomials over the evaluation domain H via oblivious NTT, and then commits to them under the KZG scheme; the resulting public commitments seed a Fiat-Shamir transcript from which random challenges β and γ are derived. Using these challenges, the provers compute the grand product polynomial $Z(X)$, which telescopes to 1 over the usable rows if and only if $\mathbf{f} \subseteq \mathbf{t}$, encoding the lookup relation as a single polynomial identity. The combined constraint polynomial $C(X)$ packages all five algebraic conditions—the grand-product recurrence, the subset alignment, the initial conditions, and the closure check—into one vanishing polynomial; dividing by $Z_H(X) = X^L - 1$ yields the quotient $Q(X)$, whose commitment is added to the transcript to derive the final evaluation challenge ζ . Finally, all polynomials are opened at ζ via batched KZG proofs, yielding a compact, publicly verifiable proof π . Throughout, every operation on secret data is performed via ABB calls; only the polynomial commitments, field evaluations, and KZG opening proofs are revealed as public outputs, making $\Pi_{\text{PermVanish}}$ oblivious by construction.

E.2 Performance Analysis

In total, permutation and vanishing arguments requires $O(N)$ $\mathcal{F}_{\text{Mult}}$, $O(N)$ $\mathcal{F}_{\text{Inverse}}$, and $O(N \log N)$ $\mathcal{F}_{\text{Add}}$. The performance analysis is shown below.

Algorithm 6: MPlookup Permutation and Vanishing Arguments $\Pi_{\text{PermVanish}}$

Input: Public table vector $\mathbf{t}$, secret-shared query vector $\mathbf{f}$, secret-shared permutation vectors $\mathbf{t}'$ and $\mathbf{f}'$

Output: Public NIZK Proof π

- 1 *Padding and encoding.* Extend all four evaluation vectors to length L by appending zeros for the $L_u - N$ excess usable rows and random field elements for the remaining $\kappa + 1$ rows, giving L evaluation points in total. Interpolate $f(X), t(X), f'(X), t'(X)$ over H via oblivious INTT on the secret-shared evaluation vectors.
- 2 *Commit and challenge.* Jointly compute KZG commitments over secret-shared coefficients:

$$f_{\text{Com}}, t_{\text{Com}}, f'_{\text{Com}}, t'_{\text{Com}} \leftarrow \text{KZG.Commit}(\text{ck}, \cdot).$$

The commitment group elements are public outputs. Include them into a Fiat-Shamir transcript $\mathcal{T}$ and derive challenges $\beta, \gamma \leftarrow \mathcal{T}$.

- 3 *Grand product.* Set $Z(\omega^0) = 1$ and compute, for $i = 0, \dots, L_u - 1$,

$$Z(\omega^{i+1}) = Z(\omega^i) \cdot \frac{(f(\omega^i) + \beta)(t(\omega^i) + \gamma)}{(f'(\omega^i) + \beta)(t'(\omega^i) + \gamma)}.$$

If $\mathbf{f} \subseteq \mathbf{t}$, the product telescopes to $Z(\omega^{L_u}) = 1$. Interpolate $Z(X)$ over H ; jointly compute

$Z_{\text{Com}} \leftarrow \text{KZG.Commit}(\text{ck}, Z)$ and include Z_{Com} into $\mathcal{T}$.

- 4 *Quotient.* Form the combined constraint polynomial

$$\begin{aligned} C(X) = & (1 - q_{\text{last}} - q_{\text{blind}}) [Z(\omega X)(f'(X) + \beta)(t'(X) + \gamma) \\ & - Z(X)(f(X) + \beta)(t(X) + \gamma)] \\ & + (1 - q_{\text{last}} - q_{\text{blind}})(f'(X) - t'(X))(f'(X) - f'(\omega^{-1}X)) \\ & + \ell_0(X)(f'(X) - t'(X)) \\ & + \ell_0(X)(1 - Z(X)) \\ & + q_{\text{last}}(X)(Z(X)^2 - Z(X)). \end{aligned}$$

The five terms enforce, respectively: (1) the grand-product recurrence on usable rows; (2) the subset condition $f'_i = t'_i$ or $f'_i = f'_{i-1}$ on usable rows; (3) the initial alignment $f'(1) = t'(1)$; (4) the initial condition $Z(1) = 1$; and (5) the closure $Z(\omega^{L_u}) = \{0, 1\}$. Since $C(X)$ vanishes on H , divide to obtain $Q(X) = C(X)/Z_H(X)$ where $Z_H(X) = X^L - 1$. Jointly compute $Q_{\text{Com}} \leftarrow \text{KZG.Commit}(\text{ck}, Q)$; include Q_{Com} into $\mathcal{T}$; derive evaluation point $\zeta \leftarrow \mathcal{T}$.

- 5 *Open.* Jointly compute polynomial evaluations $\hat{f} = f(\zeta)$, $\hat{t} = t(\zeta)$, $\hat{f}' = f'(\zeta)$, $\hat{t}' = t'(\zeta)$, $\hat{Z} = Z(\zeta)$, $\hat{Q} = Q(\zeta)$, $\hat{Z}_{\omega} = Z(\omega\zeta)$, $\hat{f}'_{\omega^{-1}} = f'(\omega^{-1}\zeta)$, and generate KZG opening proofs: π_{ζ} is a batched proof covering f, t, f', t', Z, Q at ζ ; $\pi_{\omega\zeta}$ covers Z at $\omega\zeta$; and $\pi_{\omega^{-1}\zeta}$ covers f' at $\omega^{-1}\zeta$. All evaluations and opening proofs are public outputs.
 - 6 The proof is $\pi = (f_{\text{Com}}, t_{\text{Com}}, f'_{\text{Com}}, t'_{\text{Com}}, Z_{\text{Com}}, Q_{\text{Com}}, \hat{f}, \hat{t}, \hat{f}', \hat{t}', \hat{Z}, \hat{Z}_{\omega}, \hat{f}'_{\omega^{-1}}, \hat{Q}, \pi_{\zeta}, \pi_{\omega\zeta}, \pi_{\omega^{-1}\zeta})$, where $\hat{\cdot}$ denotes the evaluation at ζ . Every element of π is a public value: commitments are group elements, evaluations are field elements, and opening proofs contain no secret shares.
-

Padding and encoding. Each of the four vectors $\mathbf{f}, \mathbf{t}, \mathbf{f}', \mathbf{t}'$ is padded to length L and interpolated via INTT. INTT twiddle factors are public roots of unity, so every butterfly requires only a local scalar multiplication and an $\mathcal{F}_{\text{Add}}$; no $\mathcal{F}_{\text{Mult}}$ is needed. The total cost is $O(N \log N) \mathcal{F}_{\text{Add}}$.

Commit and challenge. Each prover commits to its shares of the four polynomials $f(X), t(X), f'(X), t'(X)$ using group scalar multiplications with the public CRS; no $\mathcal{F}_{\text{Mult}}$ is needed. The Fiat-Shamir challenges β, γ are derived by hashing the public commitments, which is a local computation with no ABB operations.

Grand product. In each of the $L_u = O(N)$ usable rows, the recurrence $Z(\omega^{i+1}) = Z(\omega^i) \cdot \frac{(f_i + \beta)(t_i + \gamma)}{(f'_i + \beta)(t'_i + \gamma)}$ requires: one $\mathcal{F}_{\text{Mult}}$ for $(f'_i + \beta)(t'_i + \gamma)$, where both factors involve secrets while β, γ are public; one $\mathcal{F}_{\text{Inverse}}$ on the denominator; one $\mathcal{F}_{\text{Mult}}$ to form the ratio; and one $\mathcal{F}_{\text{Mult}}$ to update the running product. The numerator $(f_i + \beta)(t_i + \gamma)$ costs no $\mathcal{F}_{\text{Mult}}$ because $t_i + \gamma$ is a public scalar. Interpolating $Z(X)$ via INTT adds $O(N \log N) \mathcal{F}_{\text{Add}}$ and no $\mathcal{F}_{\text{Mult}}$. Total: $O(N) \mathcal{F}_{\text{Mult}}$ and $O(N) \mathcal{F}_{\text{Inverse}}$.

Constraint polynomial and quotient. Forming $C(X)$ requires evaluating its five terms at each point of H . The dominant cost comes from quadratic products between secret-shared polynomials: term 1 involves $Z(\omega X) \cdot (f' + \beta) \cdot (t' + \gamma)$, requiring two $\mathcal{F}_{\text{Mult}}$ per row since β, γ are public, and $Z(X) \cdot (f + \beta) \cdot (t + \gamma)$, requiring one $\mathcal{F}_{\text{Mult}}$ per row since $t + \gamma$ is public; term 2 requires one $\mathcal{F}_{\text{Mult}}$; term 5 requires one $\mathcal{F}_{\text{Mult}}$ for Z^2 . Terms 3 and 4 involve only public selectors. Dividing by $Z_H(X)$ over an extended evaluation domain uses NTT with public twiddle factors: $O(N \log N) \mathcal{F}_{\text{Add}}$, no $\mathcal{F}_{\text{Mult}}$. Total: $O(N) \mathcal{F}_{\text{Mult}}$.

Polynomial evaluations at ζ . Each polynomial evaluation $p(\zeta) = \sum_j p_j \zeta^j$ has secret coefficients p_j and public powers ζ^j , so it costs $O(N) \mathcal{F}_{\text{Add}}$ and no $\mathcal{F}_{\text{Mult}}$. Eight polynomials give $O(N) \mathcal{F}_{\text{Add}}$ in total.

KZG commitments and opening proofs. Each prover computes its partial commitment $\sum_j p_j^{(i)} \cdot [\tau^j]_1$ from its own coefficient shares and the public CRS, using only group scalar multiplications. KZG opening proofs divide the witness polynomial by a public $(X - \zeta)$ locally, then commit. No $\mathcal{F}_{\text{Mult}}$ or inter-party ABB communication is needed.

Besides, since proof π contains constant number of elements, our MPlookup protocol has succinctness. A multi-party lookup argument protocol is succinct if the proof size $|\pi|$ is $o(|R|)$, where $|R|$ denotes the size of relation R .

E.3 Proof of Security Analysis Theorem 5.5

PROOF. We classify every operation in $\Pi_{\text{PermVanish}}$.

(1) *ABB operations.* All arithmetic on secret-shared polynomial coefficients or evaluation vectors, including INTT, the grand-product recurrence, quotient computation, and polynomial evaluations at ζ , invokes $\mathcal{F}_{\text{ABB}}$ primitives. By the ideal-world security of $\mathcal{F}_{\text{ABB}}$, no collusion of at most t corrupted provers learns anything about honest provers' shares. INTT/NTT twiddle factors are public roots of unity and introduce no secret-dependent communication.

(2) *KZG commitments.* Each partial commitment $\sum_j p_j^{(i)} \cdot [\tau^j]_1$ is a local group scalar multiplication using each prover's own coefficient shares and the public CRS; it requires no communication. The aggregate commitment is reconstructed by summing partial

commitments, computable via addition on group elements without further ABB interaction. By the KZG computational hiding property [10] and the presence of $\kappa \geq 1$ blinding rows, each published group element is computationally indistinguishable from a commitment to an independent random polynomial of the same degree.

(3) *Fiat-Shamir challenges*. The challenges β, γ, ζ are derived by hashing the published commitments; they are a deterministic function of public data and depend on no secret shares.

(4) *Control flow*. All loop bounds depend only on the public domain size L ; there is no secret-dependent branching.

We construct a simulator Sim that replaces every ABB call with a fresh ideal-world invocation of $\mathcal{F}_{\text{ABB}}$ and produces KZG commitments to freshly sampled random polynomials sharing the same degree and blinding structure. Since all KZG commitments are computationally hiding and all ABB calls produce indistinguishable shares, the sequence and distribution of messages in the simulated view are computationally indistinguishable from the real view. By the UC composition theorem [33], $\Pi_{\text{PermVanish}}$ in the $\mathcal{F}_{\text{ABB}}$ -hybrid model computationally realizes $\mathcal{F}_{\text{PermVanish}}$, establishing obliviousness. $\square$

F Strawman Multi-Party Lookup Argument Preprocessing Algorithm

This section presents the complete strawman $O(N^2)$ multi-party lookup argument preprocessing algorithm in Algorithm 7, which is the fully expanded version of the abbreviated outline given in Figure 2. The strawman directly realizes the three preprocessing constraints, namely Constraints 3.1–3.3, without any algorithmic optimization, and serves as the correctness reference baseline against which MPlookup is validated. We describe each step in detail below.

Step 1: Oblivious sort on $\mathbf{f}$. The algorithm begins by applying an oblivious sort to the secret-shared query vector $\mathbf{f}$ to produce the sorted vector $\mathbf{f}'$. Sorting brings identical elements into adjacent positions, which is a prerequisite for efficiently detecting first occurrences in Step 2. Because $\mathbf{f}$ is secret-shared, the sort must be *oblivious*: the sequence of memory accesses and comparisons must be independent of the input values. An oblivious sort costs $O(N \log^2 N)$ ABB comparison operations.

Step 2: Identify first-occurrence elements in $\mathbf{f}'$. After sorting, repeated values in $\mathbf{f}'$ form contiguous runs. The first-occurrence indicator vector $\mathbf{o}$ is defined as follows: $o_1 = 1$ since the first element is trivially the first occurrence of its value, and for each subsequent position $i \in [2, M]$, $o_i = 1 - \text{Eq}(f'_i, f'_{i-1})$. Here, Eq denotes the ABB equality-testing functionality $\mathcal{F}_{\text{Eq}}$. When f'_i equals its predecessor, $o_i = 0$, marking it as a duplicate; otherwise, $o_i = 1$, marking it as a first occurrence. This step requires $M - 1$ invocations of $\mathcal{F}_{\text{Eq}}$ and runs in $O(N)$.

Step 3: Initialize $\mathbf{t}'$ with first-occurrence entries from $\mathbf{f}'$. The output vector $\mathbf{t}'$ is initialized position by position. For each index $i \leq M$, the entry is set to $t'_i \leftarrow o_i \cdot f'_i$: if $o_i = 1$, indicating a first occurrence, then t'_i receives the value f'_i ; if $o_i = 0$, indicating a duplicate, then $t'_i = 0$ serves as a placeholder that the position still needs to be filled. Positions $i > M$ also receive 0. Multiplying by o_i is a single ABB scalar multiplication per position, so this step costs $O(N)$.

Algorithm 7: Strawman $O(N^2)$ Multi-Party Lookup Argument Preprocess (Full)

Input: Secret-shared query vector $\mathbf{f}$ and table vector $\mathbf{t}$, where $M = |\mathbf{f}| \leq N = |\mathbf{t}|$. $\mathbf{t}$ contains unique elements, whereas $\mathbf{f}$ may contain duplicates.

Output: Secret-shared permuted vectors $\mathbf{f}'$ and $\mathbf{t}'$ satisfying Constraint 3.1, Constraint 3.2, and Constraint 3.3

```

// 1. Oblivious sort on  $\mathbf{f}$   $(O(N \log^2 N))$ 
1  $\mathbf{f}' \leftarrow \text{Sort}(\mathbf{f})$ 
// 2. Identify first occurrence elements in  $\mathbf{f}'$  via adjacent
   comparisons  $(O(N))$ 
2  $\mathbf{o} \leftarrow (1) \parallel \{1 - \text{Eq}(f'_i, f'_{i-1})\}_{i=2}^M$ 
// 3. Initialize  $\mathbf{t}'$  by filling first occurrence elements in  $\mathbf{f}'$ 
    $(O(N))$ 
3 for  $i \leftarrow 1$  to  $N$  do
4   if  $i \leq m$  then  $t'_i \leftarrow o_i \cdot f'_i$ 
5   else  $t'_i \leftarrow 0$ 
// 4. Mark used elements in  $\mathbf{t}$  via a nested loop  $(O(N^2))$ 
6  $\mathbf{o} \leftarrow \mathbf{1}^{1 \times n}$   $//$  Initialize all elements as unused
7 for  $i \leftarrow 1$  to  $M$  do
8    $\text{found} \leftarrow 0$ 
9   for  $j \leftarrow 1$  to  $N$  do
10     $\text{match} \leftarrow o_i \cdot o_j \cdot \text{Eq}(t_j, f'_i)$ 
11     $\text{update} \leftarrow (1 - \text{found}) \cdot \text{match}$ 
12     $o_j \leftarrow o_j \cdot (1 - \text{update})$ 
13     $\text{found} \leftarrow \text{found} + \text{update}$ 
// 5. Fill remaining positions in  $\mathbf{t}'$  with unused elements
   from  $\mathbf{t}$   $(O(N^2))$ 
14 for  $i \leftarrow 1$  to  $N$  do
15    $\text{found} \leftarrow 0$ 
16    $\text{need\_fill} \leftarrow 1 - o_i$ 
17   for  $j \leftarrow 1$  to  $N$  do
18     $\text{avail} \leftarrow \text{need\_fill} \cdot o_j$ 
19     $\text{update} \leftarrow (1 - \text{found}) \cdot \text{avail}$ 
20     $o_j \leftarrow o_j \cdot (1 - \text{update})$ 
21     $t'_i \leftarrow t'_i + (\text{update} \cdot t_j)$ 
22     $\text{found} \leftarrow \text{found} + \text{update}$ 
23    $o_i \leftarrow o_i + \text{found}$ 
24 return  $\mathbf{f}', \mathbf{t}'$ 

```

Step 4: Mark matched entries in $\mathbf{t}$ via a nested loop. At this point, the occupied positions of $\mathbf{t}'$, those with $o_i = 1$ as established in Step 2, already hold their correct values. To maintain the invariant that $\mathbf{t}'$ is a permutation of $\mathbf{t}$, the algorithm must identify which element of $\mathbf{t}$ corresponds to each occupied position, and mark it as used so it will not be reused. A fresh binary indicator vector $\mathbf{o}$ of length N is initialized to all ones, signifying that all table entries are initially available. For each outer-loop index i from 1 to M where $o_i = 1$, meaning f'_i is a first-occurrence element, the inner loop scans every table index j from 1 to N : it computes $\text{match} \leftarrow o_i \cdot o_j \cdot \text{Eq}(t_j, f'_i)$ to check whether t_j equals f'_i and is still available. Because the computation must remain oblivious, early termination upon finding the matching entry is not permitted;

the inner loop runs to completion for every i . The boolean flag found ensures that only the *first* matching table entry is consumed: $\text{update} \leftarrow (1 - \text{found}) \cdot \text{match}$ equals 1 on exactly the first match encountered, after which o_j is cleared and found is set to 1. Since $\mathbf{t}$ contains unique elements, exactly one match exists for each first-occurrence value. The overall cost of this step is $O(N^2)$ invocations of $\mathcal{F}_{\text{Eq}}$.

Step 5: Fill remaining positions in $\mathbf{t}'$ with unused elements from $\mathbf{t}$. After Step 4, the positions of $\mathbf{t}'$ that still hold 0, corresponding to duplicate values or indices beyond M , must be assigned the remaining unused entries of $\mathbf{t}$. For each position i from 1 to N , the algorithm computes $\text{need_fill}_i \leftarrow 1 - o_i$, where o_i is the post-Step-3 indicator that equals 1 for occupied positions and 0 for empty ones. When $\text{need_fill}_i = 1$, the inner loop scans $\mathbf{t}$ to find the first still-available entry: $\text{avail} \leftarrow \text{need_fill}_i \cdot o_j$ and $\text{update} \leftarrow (1 - \text{found}) \cdot \text{avail}$. Upon finding it, t'_i is incremented by $\text{update} \cdot t_j$, the entry is marked as consumed by setting $o_j \leftarrow o_j \cdot (1 - \text{update})$, and found is set to prevent further updates within the same outer iteration. As in Step 4, obliviousness forbids early termination, yielding $O(N^2)$ cost for this step as well. Upon completion, every position of $\mathbf{t}'$ has been assigned a unique element from $\mathbf{t}$, so $\mathbf{t}'$ is a permutation of $\mathbf{t}$, satisfying Constraint 3.2.

G Experiment Instructions

This appendix provides step-by-step instructions for reproducing the experimental results reported in Section 6.

G.1 System Requirements

A machine with the following minimum specifications is required:

- Operating System: Debian 13 or Ubuntu Server 24.04 LTS (amd64)
- CPU: 8 cores (4 cores per party)
- Memory: 16 GB
- Rust nightly toolchain version 1.90.0.

G.2 Run the Experiments

The experiment script `test.bash` contains the following configurable parameters, which should be set prior to running:

```
cd mpc-lookup/
nano test.bash
```

The content of `test.bash`:

```
parties=2
test_naive=0

n=8
max_n=1024
```

Below explains the parameters:

- `parties`: corresponds to n defined in the paper, the number of multi-party computation parties, i.e., provers.
- `test_naive`: whether to test the naive implementation. 0 means test MPlookup; 1 means test the strawman.
- `n`: corresponds to N defined in the paper, the initial size of the input vector t . The script will double the size of t in each iteration until it reaches `max_n`. Must be a power of 2.
- `max_n`: the maximum size of the input vector t . Must be a power of 2.

The script may then be executed. The use of `tmux` is recommended, as the experiments may require substantial time to complete.

```
bash test.bash
```

In the end, the script will save the log files to the `mpc-lookup/results` folder. Each log file contains the running time of each step of the MPlookup protocol, which can be used to draw the experiment figures in the Evaluation section.

G.3 Run the Evaluation Data Generation and Figure Drawing Scripts

To generate the evaluation data or draw figures used in the Evaluation section, run the following commands:

A Python virtual environment must first be configured, with the required dependencies installed as follows:

```
cd mpc-lookup/result-analyze/
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

The following scripts can then be executed to compute evaluation data and draw figures:

```
python compute-evaluation-data.py
python draw.py
```